

XPLORE'26 · LICET

Building the Treasure Hunt

Four browser puzzles for a live event — a cipher, a colour grid, a circuit and a 3D room — every file, every decision, and an honest account of what broke.

Event	XPLORE'26 Treasure Hunt, LICET Chennai
Format	Four web puzzles, all open at once, points with hints
Stack	Next.js 16 (App Router), React 19, TypeScript, React Three Fiber, MongoDB / Azure Cosmos DB
Sits on	The existing event platform — its pipeline and hunt grader were NOT modified
Audience	Student builders implementing this from scratch
Status	40 unit tests, 19 integration checks, all passing

Contents

1 What you are building

- 1.1 The shape of it
- 1.2 The one thing that shipped broken

2 What already existed

- 2.1 The grader you do not write
- 2.2 Cosmos is not MongoDB

3 The puzzle logic

- 3.1 Caesar shift
- 3.2 The 8×8 colour grid
- 3.3 The circuit, and four rounds of being wrong

4 Content, codes, and the seed

- 4.1 Two files, split on purpose
- 4.2 Seeding

5 The API

- 5.1 Progress, and the fix that saves the event
- 5.2 Hints

6 The shell and the puzzle contract

- 6.1 The contract
- 6.2 The shell
- 6.3 Making the contract enforceable

7 The flat puzzles

- 7.1 Caesar Cipher
- 7.2 64 Grid
- 7.3 Octavius Circuit

8 The Mystery Room

- 8.1 The manifest, and why the art can arrive later
- 8.2 The room
- 8.3 Props, and failing loudly

9 Verification

- 9.1 Running it
- 9.2 Checks that can actually fail
- 9.3 Leave the database as you found it

10 What went wrong

- 10.1 Nobody could play
- 10.2 Four leaks of the same shape
- 10.3 The scramble that took four rounds
- 10.4 Hints that cost 40 points to mislead you

10.5 Smaller ones, and what they share

11 Running it

11.1 Before the doors open

11.2 When something breaks mid-event

12 Appendix — the remaining source

12.1 `src/lib/hunt/grid.test.ts`

12.2 `src/lib/hunt/content.test.ts`

12.3 `src/app/hunt/page.tsx`

1 What you are building

Four puzzles that run in a browser, during a live event, while teams move around a campus. A cipher, an 8×8 colour grid, a wire-rotation circuit, and a 3D room you look around and pick things up in.

The thing worth understanding before you write anything is how little of this is new. The platform already had a submission pipeline, a scoring ledger, sessions, and a hunt grader that compares hashed answers and deducts hint costs. **None of it was modified.** Four puzzles with a 3D scene were added without touching a line of the pipeline.

That is the test of whether a boundary is in the right place, and it is the single most useful thing to copy from this build.

1.1 The shape of it

PUZZLE	MECHANIC	REVEALS
Caesar Cipher	Drag a shift dial; the plaintext re-renders live	A code in the decrypted text
64 Grid	Three equations pick a colour; unscramble that colour's eight letters	The word
Octavius Circuit	Rotate tiles to route power from source to sink	A code when the circuit closes
Mystery Room	Look around a 3D room, pick up five props, read the fragments	The assembled code

All four unlock at once. There is no chain: teams take whichever station is free, which is what stops four teams queueing behind the slowest puzzle. Each is worth 100 points; hints cost 15 then 25.

WHY NO CHAIN

A chain reads better on paper — solve this, unlock that. In a room with four teams and four stations it means three teams standing still. The platform's `gradeHunt` supports chaining through `nextSlug`; this event simply leaves it unset.

1.2 The one thing that shipped broken

Before the design, the outcome — because it is the most useful thing in this document.

The build passed nine per-task code reviews. Every task was correct against its own brief. Then a whole-branch review ran and found that **no team could play at all**: fifteen real teams in the live database had zero progress rows between them, and every one of them would have opened the page to a spinner that never resolved, with no error shown.

The cause was a seam. Progress rows were created in one place (`/api/enter`) and deleted in another (`scripts/seed-hunt.ts` , which wipes them on every reseed). Nothing put them back. One task owned the seed, another owned entry, and *nobody owned the lifecycle between them*.

WHAT TO TAKE FROM THIS

Reviewing each piece against its own specification cannot find a bug that lives in the gap between two pieces.

Something has to look at the whole thing, and it has to look at the running system rather than the source — the fault was invisible in code review and obvious in one database query.

Part 10 catalogues this and the others. Read it before you build, not after.

2 What already existed

The hunt is four answer-producing interfaces on top of machinery that was already there. Read this part before writing a puzzle, or you will rebuild things that exist.

2.1 The grader you do not write

Hashed answer compare, hint costs deducted from the award, and a server-side gate so a team cannot submit against a puzzle they have not unlocked. This file was not touched during the entire build.

src/lib/graders/hunt.ts

49 lines

```
1 import { hashAnswer } from "@lib/auth/session";
2 import { collections } from "@lib/db/client";
3 import type { GradeInput, GradeResult } from "./types";
4
5 /**
6  * HUNT - hashed answer compare, then unlock the next clue in the chain.
7  *
8  * The chain is enforced SERVER-side via hunt_progress: a team can only submit
9  * against a clue they've actually unlocked. Without that, anyone could read
10 * the clue slugs out of the client bundle and jump to the last one.
11 */
12 export async function gradeHunt(input: GradeInput): Promise<GradeResult> {
13   const { challenge, teamId, payload } = input;
14   const progress = await collections.huntProgress();
15
16   const current = await progress.findOne({ teamId, challengeSlug: challenge.slug });
17   if (!current) {
18     return { correct: false, points: 0, meta: { reason: "clue-not-unlocked" } };
19   }
20   if (current.solvedAt) {
21     return { correct: false, points: 0, meta: { reason: "already-solved" } };
22   }
23
24   if (hashAnswer(payload) !== challenge.config.answerHash) {
25     return { correct: false, points: 0 };
26   }
27
28   // Hints are a point cost, not a lockout - never let the award go negative.
29   const hintCosts = challenge.config.hintCosts ?? [];
30   const spent = hintCosts.slice(0, current.hintsUsed).reduce((a, b) => a + b, 0);
31   const points = Math.max(0, challenge.points - spent);
32
33   await progress.updateOne(
34     { teamId, challengeSlug: challenge.slug },
35     { $set: { solvedAt: input.receivedAt } }
36   );
37
38   // Unlock the next clue. upsert so a replayed request can't create duplicates.
39   const next = challenge.config.nextSlug;
40   if (next) {
41     await progress.updateOne(
42       { teamId, challengeSlug: next },
43       { $setOnInsert: { teamId, challengeSlug: next, unlockedAt: input.receivedAt, solvedAt: null, hintsUsed: 0 } },
44       { upsert: true }
45     );
46   }
47 }
```

```
48   return { correct: true, points, meta: { hintsUsed: current.hintsUsed, nextSlug: next ?? null } };
49 }
```

TWO DETAILS WORTH STEALING

The answer is never stored. Only `hashAnswer(code)` reaches the database. Anyone with database access still cannot read the answers.

Hints are a cost, not a lockout. The award is `Math.max(0, points - spent)` — a team that buys every hint still scores something and keeps playing, rather than being punished into standing still for forty minutes.

2.2 Cosmos is not MongoDB

The database speaks the MongoDB wire protocol and is not MongoDB. Two of its quirks bit this build directly.

WHAT YOU SEE	WHAT IT IS
--------------	------------

Error 16500	Throttling, not failure. You exceeded provisioned RU/s. Retry it. This appeared while deleting 55 test teams in one call — the helper below exists precisely for it, and the throwaway cleanup script that hit the error had not used it.
-------------	--

A unique index will not create	Only permitted on an empty collection. The coin system is keyed by <code>_id</code> for this reason — uniqueness Cosmos will not give you as a constraint, obtained by making the unique thing the key.
--------------------------------	---

src/lib/db/retry.ts

54 lines

```
1  /**
2   * Retry around Cosmos DB throughput throttling.
3   *
4   * Cosmos's RU-based Mongo API answers error 16500 when a request would exceed
5   * the account's provisioned throughput, with a `RetryAfterMs` hint in the
6   * message. It is NOT a failure – it means "ask again shortly". A real MongoDB
7   * server never does this, so nothing in the driver handles it: `retryWrites`
8   * has to be off against Cosmos anyway (see `client.ts`), so an unretried 16500
9   * surfaces to the caller as a hard error.
10  *
11  * That matters most at exactly the wrong moment. This platform's whole premise
12  * is 500 people acting inside the same five seconds, and a burst like that on a
13  * modest RU allocation is precisely when 16500 appears – so a submission that
14  * should have been accepted would come back as a 500 to a participant.
15  *
16  * Wrap bulk or burst-prone database work in this. It is not a substitute for
17  * provisioning enough throughput for the event; it's what stops a brief spike
18  * from becoming a lost submission.
19  */
20
21  const THROTTLE_CODE = 16500;
22
23  function retryAfterMs(err: unknown, attempt: number): number | null {
24    const e = err as { code?: number; errorResponse?: { errmsg?: string }; message?: string };
25    if (e?.code !== THROTTLE_CODE) return null;
26
27    // Cosmos puts the hint in the message: "Error=16500, RetryAfterMs=8, ..."
28    const text = e.errorResponse?.errmsg ?? e.message ?? "";
29    const match = /RetryAfterMs=(\d+)/.exec(text);
30    const hinted = match ? Number(match[1]) : 0;
31
32    // Honour the hint but never hammer: back off geometrically as attempts climb.
33    return Math.max(hinted, 25) * Math.pow(2, attempt);
34  }
35
```

```

36 export async function withThrottleRetry<T>(  

37   fn: () => Promise<T>,  

38   attempts = 8  

39 ): Promise<T> {  

40   let lastError: unknown;  

41  

42   for (let attempt = 0; attempt < attempts; attempt++) {  

43     try {  

44       return await fn();  

45     } catch (err) {  

46       const wait = retryAfterMs(err, attempt);  

47       if (wait === null) throw err; // not throttling - a real failure  

48       lastError = err;  

49       await new Promise((r) => setTimeout(r, wait));  

50     }  

51   }  

52  

53   throw lastError;  

54 }

```

DO NOT ADD RETRYWRITES

Cosmos rejects `retryWrites=true`, and it rejects it on *write* rather than on *connect* — so everything looks healthy until the first team submits an answer.

3 The puzzle logic

All three of the flat puzzles keep their logic in framework-free modules under `src/lib/hunt/`, unit-tested with Vitest. The React components render state and forward clicks; they contain no rules.

This is not tidiness. The existing verification script needs a running server and a live database, which is fine for integration and useless as a development loop. Pure modules can be tested in milliseconds, and the circuit's connectivity bug took four rounds to kill precisely because it needed thousands of seeded runs to reproduce.

3.1 Caesar shift

The smallest piece, and worth reading for the property it has: the seed generates the ciphertext from the plaintext using the same function the UI decrypts with, so the puzzle and its answer cannot drift apart.

`src/lib/hunt/caesar.ts`

12 lines

```
1 /**
2  * Caesar shift. Pure, so the puzzle UI can call it on every drag frame and the
3  * seed script can call it to produce the ciphertext from the plaintext – one
4  * implementation, so the puzzle and its answer cannot drift apart.
5  */
6 export function shift(text: string, by: number): string {
7   const n = ((by % 26) + 26) % 26; // negatives and >26 both normalise here
8   return text.replace(/[a-z]/gi, (ch) => {
9     const base = ch <= "Z" ? 65 : 97;
10    return String.fromCharCode(((ch.charCodeAt(0) - base + n) % 26) + base);
11  });
12 }
```

`src/lib/hunt/caesar.test.ts`

30 lines

```
1 import { describe, expect, it } from "vitest";
2 import { shift } from "../caesar";
3
4 describe("shift", () => {
5   it("shifts letters forward and wraps past z", () => {
6     expect(shift("abc xyz", 3)).toBe("def abc");
7   });
8
9   it("preserves case", () => {
10    expect(shift("AbZ", 1)).toBe("BcA");
11  });
12
13   it("leaves non-letters untouched", () => {
14     expect(shift("a-b 1!", 1)).toBe("b-c 1!");
15   });
16
17   it("round-trips: shifting by n then by 26-n returns the original", () => {
18     const text = "the daily bugle";
19     expect(shift(shift(text, 7), 19)).toBe(text);
20   });
21
22   it("treats a shift of 0 and 26 as identity", () => {
23     expect(shift("spider", 0)).toBe("spider");
24     expect(shift("spider", 26)).toBe("spider");
25   });
26 }
```

```

26
27   it("accepts negative shifts", () => {
28     expect(shift("def", -3)).toBe("abc");
29   });
30 });

```

3.2 The 8×8 colour grid

Eight colours, eight cells each, each colour's letters anagramming to one word. Solve three equations, take the sum modulo 8, and that selects the colour you unscramble.

src/lib/hunt/grid.ts

59 lines

```

1  /**
2   * The 8x8 colour grid.
3   *
4   * Eight colours, eight cells each, each colour's letters anagramming to one
5   * word. All eight groups spell real words so the grid gives no clue which one
6   * the equations will select - if only the target group were a word, a team
7   * could skip the equations entirely by looking for the group that reads.
8   */
9   export const GRID_COLOURS = [
10    "#e63946", "#f4a261", "#e9c46a", "#2a9d8f",
11    "#4361ee", "#7209b7", "#f72585", "#d8e2dc",
12  ] as const;
13
14  export interface GridCell {
15    letter: string;
16    colour: number;
17  }
18
19  /** Mulberry32 - small deterministic PRNG so a seed always rebuilds the same grid. */
20  function rng(seed: number): () => number {
21    let a = seed >>> 0;
22    return () => {
23      a = (a + 0x6d2b79f5) >>> 0;
24      let t = Math.imul(a ^ (a >>> 15), 1 | a);
25      t = (t + Math.imul(t ^ (t >>> 7), 61 | t)) ^ t;
26      return ((t ^ (t >>> 14)) >>> 0) / 4294967296;
27    };
28  }
29
30  export function buildGrid(words: string[], seed: number): GridCell[] {
31    if (words.length !== GRID_COLOURS.length) {
32      throw new Error(`buildGrid needs exactly ${GRID_COLOURS.length} words, got ${words.length}`);
33    }
34    for (const w of words) {
35      if (w.length !== 8) throw new Error(`Every grid word must be 8 letters: "${w}" is ${w.length}`);
36    }
37
38    const cells: GridCell[] = [];
39    words.forEach((word, colour) => {
40      for (const letter of word) cells.push({ letter: letter.toUpperCase(), colour });
41    });
42
43    // Fisher-Yates against the seeded PRNG.
44    const next = rng(seed);
45    for (let i = cells.length - 1; i > 0; i -= 1) {
46      const j = Math.floor(next() * (i + 1));
47      [cells[i], cells[j]] = [cells[j], cells[i]];
48    }
49    return cells;
50  }

```

```

51
52 export function lettersFor(grid: GridCell[], colour: number): string[] {
53   return grid.filter((c) => c.colour === colour).map((c) => c.letter);
54 }
55
56 export function isAnagram(a: string, b: string): boolean {
57   const norm = (s: string) => s.toUpperCase().replace(/^[A-Z]/g, "").split("").sort().join("");
58   return norm(a) === norm(b);
59 }

```

ALL EIGHT GROUPS SPELL REAL WORDS

If only the answer group anagrammed to something, a team could skip the equations entirely and look for the group that reads. The puzzle's gate is knowing *which* colour, so the other seven have to be equally plausible.

3.3 The circuit, and four rounds of being wrong

A grid of wire tiles; clicking rotates one 90°. The board is scrambled at seed time, and the scramble must never produce a board that is already solved — that would hand out free points with nothing erroring to say so.

That guarantee took four attempts. The full account is in part 10; the working version is here.

src/lib/hunt/circuit.ts

183 lines

```

1  /**
2   * The rotate-the-wires puzzle.
3   *
4   * Connectivity requires MUTUAL openings: tile A must open toward B and B back
5   * toward A. Checking only one direction makes half-connections count and the
6   * board lights up on a circuit that isn't closed, which reads as the puzzle
7   * being broken rather than the player being wrong.
8   */
9  export type Dir = 0 | 1 | 2 | 3; // up, right, down, left
10
11 export interface Tile {
12   kind: "straight" | "elbow" | "tee" | "source" | "sink" | "empty";
13   rot: Dir;
14 }
15
16 export interface Board {
17   w: number;
18   h: number;
19   tiles: Tile[]; // row-major, length w*h
20 }
21
22 const BASE: Record<Tile["kind"], Dir[]> = {
23   straight: [0, 2],
24   elbow: [0, 1],
25   tee: [0, 1, 3],
26   source: [0],
27   sink: [0],
28   empty: [],
29 };
30
31 export function openings(tile: Tile): Dir[] {
32   return BASE[tile.kind].map((d) => (((d + tile.rot) % 4) as Dir));
33 }
34
35 export function rotate(tile: Tile): Tile {
36   return { ...tile, rot: (((tile.rot + 1) % 4) as Dir) };
37 }
38

```

```

39 const DELTA: Record<Dir, [number, number]> = {
40   0: [0, -1], 1: [1, 0], 2: [0, 1], 3: [-1, 0],
41 };
42 const OPPOSITE: Record<Dir, Dir> = { 0: 2, 1: 3, 2: 0, 3: 1 };
43
44 export function isConnected(board: Board): boolean {
45   const { w, h, tiles } = board;
46   const start = tiles.findIndex((t) => t.kind === "source");
47   const target = tiles.findIndex((t) => t.kind === "sink");
48   if (start === -1 || target === -1) return false;
49
50   const seen = new Set<number>([start]);
51   const queue = [start];
52
53   while (queue.length) {
54     const at = queue.shift()!;
55     if (at === target) return true;
56
57     const x = at % w;
58     const y = Math.floor(at / w);
59
60     for (const dir of openings(tiles[at])) {
61       const [dx, dy] = DELTA[dir];
62       const nx = x + dx;
63       const ny = y + dy;
64       if (nx < 0 || ny < 0 || nx >= w || ny >= h) continue;
65
66       const next = ny * w + nx;
67       if (seen.has(next)) continue;
68       // Mutual: the neighbour must open back toward us.
69       if (!openings(tiles[next]).includes(OPPOSITE[dir])) continue;
70
71       seen.add(next);
72       queue.push(next);
73     }
74   }
75   return false;
76 }
77
78 function rng(seed: number): () => number {
79   let a = seed >>> 0;
80   return () => {
81     a = (a + 0x6d2b79f5) >>> 0;
82     let t = Math.imul(a ^ (a >>> 15), 1 | a);
83     t = (t + Math.imul(t ^ (t >>> 7), 61 | t)) ^ t;
84     return ((t ^ (t >>> 14)) >>> 0) / 4294967296;
85   };
86 }
87
88 /**
89  * Get the BFS path from source to sink, if one exists.
90  * Returns an array of tile indices along the connected path.
91  */
92 function getConnectedPath(board: Board): number[] {
93   const { w, h, tiles } = board;
94   const start = tiles.findIndex((t) => t.kind === "source");
95   const target = tiles.findIndex((t) => t.kind === "sink");
96   if (start === -1 || target === -1) return [];
97
98   const seen = new Set<number>([start]);
99   const queue = [start];
100   const parent: Record<number, number> = { [start]: -1 };
101
102   while (queue.length) {
103     const at = queue.shift()!;
104     if (at === target) {

```

```

105 // Reconstruct path by following parents back to source
106 const path: number[] = [];
107 let current = target;
108 while (current !== -1) {
109     path.unshift(current);
110     current = parent[current];
111 }
112 return path;
113 }
114
115 const x = at % w;
116 const y = Math.floor(at / w);
117
118 for (const dir of openings(tiles[at])) {
119     const [dx, dy] = DELTA[dir];
120     const nx = x + dx;
121     const ny = y + dy;
122     if (nx < 0 || ny < 0 || nx >= w || ny >= h) continue;
123
124     const next = ny * w + nx;
125     if (seen.has(next)) continue;
126     // Mutual: the neighbour must open back toward us.
127     if (!openings(tiles[next]).includes(OPPOSITE[dir])) continue;
128
129     seen.add(next);
130     parent[next] = at;
131     queue.push(next);
132 }
133 }
134 return [];
135 }
136
137 /**
138  * Randomise rotations, then guarantee the result is NOT already solved.
139  *
140  * A board that starts solved is the worst outcome: it looks like a bug, the
141  * team gets free points, and nothing errors to tell you it happened.
142  *
143  * Disconnection cannot be assumed from rotating one tile, even if that tile
144  * is on the path. Some board topologies have multiple paths from source to
145  * sink; rotating a single tile may not break connectivity. Instead, we try
146  * rotating tiles found on the actual connected path, using the RNG to
147  * deterministically select which tiles to try. If exhaustive rotation of path
148  * tiles still cannot disconnect, this is a content bug—a circuit board whose
149  * topology is unbreakable by any single-tile rotation—and we throw rather
150  * than silently return a solved board.
151  */
152 export function scramble(board: Board, seed: number): Board {
153     const next = rng(seed);
154     let out: Board = {
155         ...board,
156         tiles: board.tiles.map((t) =>
157             t.kind === "empty" ? { ...t } : { ...t, rot: (Math.floor(next() * 4) % 4) as Dir }
158         ),
159     };
160
161     let guard = 0;
162     while (isConnected(out) && guard < 1000) {
163         const path = getConnectedPath(out);
164         if (path.length === 0) break;
165
166         // Use RNG to pick a tile from the path to rotate, ensuring determinism.
167         const pathIdx = Math.floor(next() * path.length);
168         const targetTile = path[pathIdx];
169
170         out = { ...out, tiles: out.tiles.map((t, k) => (k === targetTile ? rotate(t) : t)) };

```

```

171     guard += 1;
172   }
173
174   if (isConnected(out)) {
175     throw new Error(
176       "Cannot disconnect board after 1000 rotation attempts: " +
177       "the circuit board's topology is unbreakable by single-tile rotation. " +
178       "This is a content-authoring bug."
179     );
180   }
181
182   return out;
183 }

```

MUTUAL OPENINGS

Connectivity requires tile A to open toward B *and* B back toward A. Checking one direction lights the board on a circuit that is not closed, which reads as a broken puzzle rather than a wrong answer.

The regression tests are worth reading in full, because two earlier versions of them passed against the bug they were written to catch:

src/lib/hunt/circuit.test.ts

174 lines

```

1  import { describe, expect, it } from "vitest";
2  import { Board, isConnected, openings, rotate, scramble, Tile } from "../circuit";
3
4  const t = (kind: Tile["kind"], rot = 0): Tile => ({ kind, rot: rot as Tile["rot"] });
5
6  /** 3x1: source -> straight -> sink, all already aligned horizontally. */
7  const SOLVED: Board = {
8    w: 3, h: 1,
9    tiles: [t("source", 1), t("straight", 1), t("sink", 3)],
10 };
11
12 describe("openings", () => {
13   it("gives a straight two opposite sides", () => {
14     expect(openings(t("straight", 0)).sort()).toEqual([0, 2]);
15     expect(openings(t("straight", 1)).sort()).toEqual([1, 3]);
16   });
17
18   it("gives an elbow two adjacent sides", () => {
19     expect(openings(t("elbow", 0)).sort()).toEqual([0, 1]);
20   });
21
22   it("gives a tee three sides", () => {
23     expect(openings(t("tee", 0)).length).toBe(3);
24   });
25
26   it("gives source and sink exactly one", () => {
27     expect(openings(t("source", 1))).toEqual([1]);
28     expect(openings(t("sink", 3))).toEqual([3]);
29   });
30
31   it("gives empty none", () => {
32     expect(openings(t("empty"))).toEqual([]);
33   });
34 });
35
36 describe("rotate", () => {
37   it("advances rotation by one quarter turn", () => {
38     expect(rotate(t("elbow", 0)).rot).toBe(1);
39   });
40 }

```

```

41   it("wraps 3 back to 0", () => {
42     expect(rotate(t("elbow", 3)).rot).toBe(0);
43   });
44
45   it("returns to the original after four turns", () => {
46     expect(rotate(rotate(rotate(rotate(t("elbow", 2))))).rot).toBe(2);
47   });
48 });
49
50 describe("isConnected", () => {
51   it("is true when source reaches sink through aligned tiles", () => {
52     expect(isConnected(SOLVED)).toBe(true);
53   });
54
55   it("is false when a middle tile is turned away", () => {
56     const broken: Board = { ...SOLVED, tiles: [t("source", 1), t("straight", 0), t("sink", 3)] };
57     expect(isConnected(broken)).toBe(false);
58   });
59
60   it("is false when two tiles touch but neither opens toward the other", () => {
61     const facing: Board = { ...SOLVED, tiles: [t("source", 0), t("straight", 1), t("sink", 3)] };
62     expect(isConnected(facing)).toBe(false);
63   });
64
65   it("requires mutual openings, not just one side", () => {
66     // straight opens left/right, sink opens left: the pair connects.
67     // Turn the sink up and the same straight no longer reaches it.
68     const up: Board = { ...SOLVED, tiles: [t("source", 1), t("straight", 1), t("sink", 0)] };
69     expect(isConnected(up)).toBe(false);
70   });
71 });
72
73 describe("scramble", () => {
74   it("never returns an already-solved board", () => {
75     for (let seed = 1; seed <= 50; seed += 1) {
76       expect(isConnected(scramble(SOLVED, seed))).toBe(false);
77     }
78   });
79
80   it("is deterministic for a seed", () => {
81     // Same seed must produce same result (deterministic).
82     const first = scramble(SOLVED, 9);
83     const second = scramble(SOLVED, 9);
84     expect(first).toEqual(second);
85     // Different seeds must produce different results (seed influences output).
86     const different = scramble(SOLVED, 10);
87     expect(first).not.toEqual(different);
88   });
89
90   it("leaves the board shape and tile kinds unchanged", () => {
91     const s = scramble(SOLVED, 3);
92     expect(s.w).toBe(SOLVED.w);
93     expect(s.h).toBe(SOLVED.h);
94     expect(s.tiles.map((x) => x.kind)).toEqual(SOLVED.tiles.map((x) => x.kind));
95   });
96
97   it("regression: does not return solved board when first non-empty tile is off the path", () => {
98     // tee is at index 0, but source->sink path is at indices 1->2.
99     // The old implementation would findIndex for first non-empty and get tee,
100     // then rotate only tee, which is off the path and never breaks the circuit.
101     const offPathBoard: Board = {
102       w: 3, h: 1,
103       tiles: [t("tee", 0), t("source", 1), t("sink", 3)],
104     };
105     for (let seed = 1; seed <= 50; seed += 1) {
106       expect(isConnected(scramble(offPathBoard, seed))).toBe(false);

```

```

107     }
108   });
109
110   // Hub topology: source at (2,2) with all 4 orthogonal neighbors opening
111   // back toward it, each reaching the shared sink at (2,0) by a route that
112   // does not depend on any other neighbor's tiles. This is the shape that
113   // defeats a buggy scramble which rotates only the source tile in its
114   // de-solve loop: the source has one opening, so rotating it cycles through
115   // 4 states, and on a hub every one of those 4 states stays connected
116   // (through whichever neighbor the source happens to face), so the guard
117   // exhausts and a solved board is returned silently.
118   //
119   // Layout (w=5, h=5, row-major, "." = empty):
120   //   . . SK . .
121   //   . E1 T1 E2 .
122   //   . T2 SRC T3 .
123   //   . E3 T4 E4 .
124   //   . . . . .
125   // SK=sink rot2, T1=tee rot1(2,1), T3=tee rot3(3,2), T4=tee rot0(2,3),
126   // T2=tee rot1(1,2), E1=elbow rot1(1,1), E2=elbow rot2(3,1),
127   // E3=elbow rot0(1,3), E4=elbow rot3(3,3). Together the four corners and
128   // four tees form a ring around the source; the ring connects to the sink
129   // above T1. Verified against the real openings()/isConnected() (not just
130   // by hand): for every source rotation 0..3, with all other tiles fixed as
131   // below, the board is connected - see the first test below.
132   const buildHubBoard = (sourceRot: 0 | 1 | 2 | 3): Board => ({
133     w: 5,
134     h: 5,
135     tiles: [
136       t("empty"), t("empty"), t("sink", 2), t("empty"), t("empty"),
137       t("empty"), t("elbow", 1), t("tee", 1), t("elbow", 2), t("empty"),
138       t("empty"), t("tee", 1), t("source", sourceRot), t("tee", 3), t("empty"),
139       t("empty"), t("elbow", 0), t("tee", 0), t("elbow", 3), t("empty"),
140       t("empty"), t("empty"), t("empty"), t("empty"), t("empty"),
141     ],
142   });
143
144   it("hub board is a genuine hub: every source rotation reaches the sink", () => {
145     // Deterministic, no seeds. This is the property attempts 2 and 3 got
146     // wrong: they shipped boards where the source's neighbors were
147     // unreachable (diagonal, or empty), so isConnected was false for every
148     // rotation and the regression test below passed against both the buggy
149     // and the correct algorithm. If this assertion fails, the board below
150     // is not a real hub and the seed sweep proves nothing.
151     for (const rot of [0, 1, 2, 3] as const) {
152       expect(isConnected(buildHubBoard(rot))).toBe(true);
153     }
154   });
155
156   it("regression: never returns a connected board on a hub topology", () => {
157     // A buggy scramble that rotates only the source tile in its de-solve
158     // loop cycles the source through all 4 rotations; on this hub board
159     // every one of those 4 states is connected (proven above), so that
160     // guard loop exhausts without ever disconnecting the board.
161     //
162     // Empirically (see task-3-report.md), that bug reproduces at ~1 in
163     // 14,500 seeds against this exact board: 138 connected-returns across
164     // 2,000,000 seeds. To make missing it a near-certainty rather than a
165     // coin flip, sweep N seeds such that  $(1 - p)^N$  is negligible:
166     //  $p \approx 1/14,493$ 
167     //  $N = 200,000 \Rightarrow (1 - p)^N \approx e^{(-200000 * 1/14493)} \approx e^{-13.8} \approx 1e-6$ 
168     // i.e. under a one-in-a-million chance of missing the bug if it were
169     // still present, while running in well under a second.
170     for (let seed = 1; seed <= 200_000; seed += 1) {
171       expect(isConnected(scramble(buildHubBoard(0), seed))).toBe(false);
172     }

```



```
173     });
```

```
174 });
```

4 Content, codes, and the seed

Where the puzzles' actual words live, how they reach the database, and the rule that governs all of it: **an answer must never travel to the browser except as something the player has to work for.**

4.1 Two files, split on purpose

`codes.ts` holds the four reveal codes and is the only hunt module a client component may import.

`content.ts` holds everything else — ciphertext, equations, hints — and is server-only.

src/lib/hunt/codes.ts

16 lines

```
1 /**
2  * The four reveal codes.
3  *
4  * Client-safe by design, not by oversight – see spec §2.1. A puzzle needs its
5  * code to know when it has been solved, and the anti-cheat model is that the
6  * team still has to type it while a coordinator watches.
7  *
8  * These are hashed by the seed into answerHash; the plaintext never enters a
9  * challenge document, so this file is the only copy that ships.
10 */
11 export const CODES = {
12   cipher: "WEBSLING",
13   grid: "DAYBUGLE",
14   circuit: "ARCLIGHT",
15   room: "ARCHIVES88",
16 } as const;
```

WHY THE SPLIT

A puzzle needs its code to know when it has been solved, so the code has to reach the browser. The hints do not. Importing `content.ts` from a component would ship the hint text into the bundle — free hints, with nothing erroring to tell you. One file that may cross the boundary, one that may not, and a lint rule enforcing it (see 6.3).

src/lib/hunt/content.ts

90 lines

```
1 import type { Board, Tile } from "../circuit";
2 import { CODES } from "../codes";
3
4 /**
5  * All four puzzles' content in one file so coordinators can reword without
6  * reading any other code. Nothing here is imported by a client component
7  * except through the seed and the shell – the reveal CODES must not reach the
8  * browser inside a challenge document.
9  */
10 export const HUNT_SLUGS = ["hunt-cipher", "hunt-grid", "hunt-circuit", "hunt-room"] as const;
11 export type HuntSlug = (typeof HUNT_SLUGS)[number];
12
13 export const CIPHER = {
14   plaintext: "the spider waits where the last light falls code websling",
15   shiftBy: 7,
16   code: CODES.cipher,
17 };
18
19 const tile = (kind: Tile["kind"], rot = 0): Tile => ({ kind, rot: rot as Tile["rot"] });
20
21 export const GRID = {
```

```

22 // Eight real words so no group visibly stands out as "the answer group".
23 words: [
24     "spiderly", "webbings", "villains", "sybiote",
25     "multiver", "gwenpool", "octopusx", "daybugle",
26 ],
27 seed: 20260728,
28 // All three are one-step arithmetic on universally-known facts (spec
29 // §5.2) – answerable from general knowledge alone, with no reference to
30 // undefined app state ("Peter's floor number", "the room") or ambiguous
31 // readings ("half the arms" – 4 mechanical or 8 counting the torso).
32 // content.test.ts asserts these numbers actually sum to targetColour mod 8.
33 equations: [
34     "Legs on a spider, minus 2",
35     "Wheels on a bicycle, plus 3",
36     "Eyes on a typical spider, divided by 2",
37 ],
38 answers: [6, 5, 4],
39 // (6 + 5 + 4) mod 8 = 7 -> colour index 7 -> "daybugle"
40 targetColour: 7,
41 code: CODES.grid,
42 };
43
44 export const CIRCUIT = {
45     board: {
46         w: 4,
47         h: 3,
48         tiles: [
49             tile("source", 1), tile("straight", 1), tile("elbow", 1), tile("empty"),
50             tile("empty"),         tile("empty"),         tile("straight", 0), tile("empty"),
51             tile("empty"),         tile("elbow", 3),       tile("elbow", 0), tile("sink", 3),
52         ],
53     } as Board,
54     seed: 4821,
55     code: CODES.circuit,
56 };
57
58 export const ROOM = {
59     clues: ["AR", "CH", "IV", "ES", "88"],
60     code: CODES.room,
61 };
62
63 export const HINTS: Record<HuntSlug, [string, string]> = {
64     "hunt-cipher": [
65         "Every letter moved the same distance down the alphabet.",
66         "The shift is a single digit, and it is odd.",
67     ],
68     "hunt-grid": [
69         "Solve all three equations, then add the results together.",
70         "Take that total modulo 8 – the remainder is the colour's position.",
71     ],
72     "hunt-circuit": [
73         "Work backwards from the sink, not forwards from the source.",
74         "One tile in the middle column only ever needs a half turn.",
75     ],
76     "hunt-room": [
77         // Both hints must describe what the room actually does – see
78         // ROOM_MANIFEST (src/lib/hunt/manifest.ts) and MysteryRoom.tsx. All five
79         // props sit in front of the fixed camera (z between -4.1 and 1.6 versus
80         // the camera at z=4.8 looking down -Z) and yaw is clamped to +/-0.85 rad,
81         // so nothing is ever behind the player – "behind you" was unreachable by
82         // construction and a hint saying otherwise sends a team looking for
83         // something that cannot exist.
84         "All five objects are somewhere in front of you – drag to look left and right, and check up on the
85         walls as well as down near the floor.",
86         // There is no manual reassembly step: MysteryRoom derives each fragment
87         // from CODES.room itself and fills the answer box once all five props

```

```

87     // are found, regardless of the order they were picked up in.
88     "You don't have to remember the order you found them in – once all five are picked up, the code
fills in the answer box for you.",
89   ],
90 };

```

4.2 Seeding

```
npx tsx --env-file=.env.local scripts/seed-hunt.ts
```

scripts/seed-hunt.ts

109 lines

```

1  /**
2   * Seed the four treasure hunt puzzles.
3   *
4   *   npx tsx --env-file=.env.local scripts/seed-hunt.ts
5   *
6   * Safe to re-run: it removes the four hunt challenges and their progress rows
7   * first, so a reseed is a clean slate rather than a duplicate set.
8   *
9   * Every puzzle stores its SOLVED-SHAPE output in config, never the raw input
10  * that produced it: the cipher stores ciphertext (not the plaintext), the
11  * circuit stores the scrambled board (not board+seed), and the grid stores
12  * the already-built, shuffled letter/colour cells (not the eight source
13  * words + seed). The grid's words are real dictionary words disguising the
14  * one that is the answer – storing `gridWords` verbatim would put that whole
15  * word in the database as a single array element, which is exactly the leak
16  * step 7 below checks for. Building the grid here and storing only the
17  * shuffled cells keeps the letters that spell the answer scattered across 64
18  * JSON entries instead of sitting together as one readable string.
19  */
20  import { hashAnswer } from "../src/lib/auth/session";
21  import { collections, ensureIndexes } from "../src/lib/db/client";
22  import { CIPHER, CIRCUIT, GRID, HINTS, HUNT_SLUGS, ROOM } from "../src/lib/hunt/content";
23  import { isConnected, scramble } from "../src/lib/hunt/circuit";
24  import { buildGrid } from "../src/lib/hunt/grid";
25  import { shift } from "../src/lib/hunt/caesar";
26
27  async function main() {
28    await ensureIndexes();
29
30    // The scrambled board must not already be solved. Assert rather than trust:
31    // a solved board hands out free points and nothing errors to tell you.
32    const scrambled = scramble(CIRCUIT.board, CIRCUIT.seed);
33    if (isConnected(scrambled)) {
34      throw new Error("CIRCUIT.seed produces an already-solved board – change the seed");
35    }
36
37    // Built once here, server-side only. The client (and the database) only
38    // ever sees this shuffled cell list, never GRID.words or GRID.seed.
39    const gridCells = buildGrid(GRID.words, GRID.seed);
40
41    const challenges = await collections.challenges();
42    const progress = await collections.huntProgress();
43
44    await challenges.deleteMany({ type: "hunt", slug: { $in: [...HUNT_SLUGS] } });
45    await progress.deleteMany({ challengeSlug: { $in: [...HUNT_SLUGS] } });
46
47    // Config objects go through named consts (rather than inline literals) so
48    // per-puzzle fields such as `ciphertext` or `gridCells` – which don't exist
49    // on every hunt challenge and so aren't part of the shared Challenge config
50    // type – don't need that type widened just to seed four rows.
51    const cipherConfig = {

```

```

52     answerHash: hashAnswer(CIPHER.code),
53     hintCosts: [15, 25],
54     ciphertext: shift(CIPHER.plaintext, CIPHER.shiftBy),
55 };
56 const gridConfig = {
57     answerHash: hashAnswer(GRID.code),
58     hintCosts: [15, 25],
59     equations: GRID.equations,
60     gridCells,
61 };
62 const circuitConfig = {
63     answerHash: hashAnswer(CIRCUIT.code),
64     hintCosts: [15, 25],
65     board: scrambled,
66 };
67 // No `clues` field here, deliberately. It used to store ROOM.clues
68 // verbatim - ["AR", "CH", "IV", "ES", "88"] - which /api/hunt/progress served
69 // to every client on page load, before the room was ever opened:
70 // `.join("")` was the whole reveal code, just fragmented across array
71 // elements rather than sitting in the database as one string. The client
72 // now derives the same fragments itself, in the browser, by splitting the
73 // already-public CODES.room constant (see MysteryRoom.tsx) - so there is
74 // nothing left for the challenge document to leak.
75 const roomConfig = {
76     answerHash: hashAnswer(ROOM.code),
77     hintCosts: [15, 25],
78 };
79
80 await challenges.insertMany([
81     {
82         type: "hunt", slug: "hunt-cipher", title: "Caesar Cipher", points: 100,
83         opensAt: null, closesAt: null,
84         config: cipherConfig,
85     },
86     {
87         type: "hunt", slug: "hunt-grid", title: "64 Grid", points: 100,
88         opensAt: null, closesAt: null,
89         config: gridConfig,
90     },
91     {
92         type: "hunt", slug: "hunt-circuit", title: "Octavius Circuit", points: 100,
93         opensAt: null, closesAt: null,
94         config: circuitConfig,
95     },
96     {
97         type: "hunt", slug: "hunt-room", title: "Mystery Room", points: 100,
98         opensAt: null, closesAt: null,
99         config: roomConfig,
100     },
101 ]);
102
103 console.log(`\n Seeded ${HUNT_SLUGS.length} hunt puzzles.`);
104 console.log(" Reveal codes are hashed - they are not in any challenge document.");
105 console.log(` Hints: ${Object.values(HINTS).flat().length} across ${HUNT_SLUGS.length}
puzzles.\n`);
106 process.exit(0);
107 }
108
109 main().catch((e) => { console.error(e); process.exit(1); });

```

THE RULE THIS FILE EXISTS TO ENFORCE

Reveal codes reach the database **only** as `hashAnswer(code)` . Everything else stored in `config` is derived: the ciphertext is the shifted plaintext, the grid is pre-shuffled cells, the board is the scrambled circuit.

An earlier version stored `gridWords` — the eight source words — which put the answer in the database in plain text. The check at the bottom of this script caught it. Keep that check.

5 The API

Two routes. One reports what a team can play; one sells hints. Both are small, and both contain a decision that took a bug to find.

5.1 Progress, and the fix that saves the event

src/app/api/hunt/progress/route.ts

86 lines

```
1 import { NextResponse } from "next/server";
2 import { ObjectId } from "mongodb";
3 import { getSession } from "@lib/auth/guard";
4 import { collections } from "@lib/db/client";
5 import { HUNT_SLUGS } from "@lib/hunt/content";
6
7 /**
8  * What this team can play, and how far they have got.
9  *
10  * answerHash is stripped before responding. It is a hash, not the code, but
11  * shipping it invites an offline dictionary attack against an eight-letter
12  * answer – which is a weekend's work, not a lifetime's.
13  */
14 export async function GET() {
15   const session = await getSession();
16   if (!session) return NextResponse.json({ error: "Not signed in" }, { status: 401 });
17
18   const teamId = new ObjectId(session.teamId);
19   const [challenges, progress] = await Promise.all([
20     collections.challenges(),
21     collections.huntProgress(),
22   ]);
23
24   // Self-healing hunt_progress. This is load-bearing, not defensive
25   // decoration – DO NOT remove it or "simplify" it into a plain insert.
26   //
27   // hunt_progress rows are otherwise created in exactly one place
28   // (/api/enter's unclaimed-coin branch) and deleted in another
29   // (scripts/seed-hunt.ts wipes them for every team on every reseed, which
30   // the spec explicitly allows mid-event). Nothing else ever backfills them.
31   // That means: a team that entered via the returning-coin or access-code
32   // branch never gets rows at all; a mid-event reseed erases solvedAt for
33   // every team that already solved something; and a transient insertMany
34   // failure after the coin+team+participant are committed strands that team
35   // hunt-less forever. All three look identical from the client: four empty
36   // puzzles that never load.
37   //
38   // Upserting here, on every load, collapses all three cases at once – any
39   // page load after the gap heals it. `$setOnInsert` is the whole point: it
40   // only sets fields when the upsert CREATES a new document, so a row that
41   // already exists is left completely untouched. Using `$set` here instead
42   // would reset solvedAt to null and hintsUsed to 0 on every page load,
43   // silently un-solving every puzzle a team has already gotten points for
44   // (and letting appendScore fire a second time), which is worse than the
45   // bug this fixes.
46   await Promise.all(
47     HUNT_SLUGS.map((challengeSlug) =>
48       progress.updateOne(
49         { teamId, challengeSlug },
50         {
51           $setOnInsert: {
```

```

52         teamId,
53         challengeSlug,
54         unlockedAt: new Date(),
55         solvedAt: null,
56         hintsUsed: 0,
57     },
58 },
59 { upsert: true }
60 )
61 )
62 );
63
64 const docs = await challenges.find({ type: "hunt", slug: { $in: [...HUNT_SLUGS] } }).toArray();
65 const rows = await progress.find({ teamId }).toArray();
66 const byslug = new Map(rows.map((r) => [r.challengeSlug, r]));
67
68 const puzzles = HUNT_SLUGS.map((slug) => {
69     const doc = docs.find((d) => d.slug === slug);
70     const row = byslug.get(slug);
71     if (!doc || !row) return null;
72
73     const { answerHash, hintCosts, ...safeConfig } = doc.config;
74     return {
75         slug,
76         title: doc.title,
77         points: doc.points,
78         solved: Boolean(row.solvedAt),
79         hintsUsed: row.hintsUsed,
80         hintCount: (hintCosts ?? []).length,
81         config: safeConfig,
82     };
83 }).filter((p): p is NonNullable<typeof p> => p !== null);
84
85 return NextResponse.json({ puzzles });
86 }

```

THE UPSERT IS LOAD-BEARING

Do not remove it or simplify it into a plain insert. Progress rows are created only in `/api/enter`'s unclaimed-coin branch and deleted by every reseed. Without this upsert: a team entering by any other route never gets rows; a mid-event reseed erases `solvedAt` for everyone; and a transient failure after the team is committed strands that team permanently. All three look identical from the client — four puzzles that never load.

`$setOnInsert` rather than `$set` is the whole point. `$set` would reset `solvedAt` and `hintsUsed` on every page load, silently un-solving puzzles a team had already scored and letting the award fire twice.

It also turns `seed-hunt.ts` from a mid-event footgun into a safe reset button, which is the answer to "what does the coordinator do when something breaks".

5.2 Hints

src/app/api/hunt/hint/route.ts

58 lines

```

1 import { NextResponse } from "next/server";
2 import { ObjectId } from "mongodb";
3 import { getSession } from "@lib/auth/guard";
4 import { collections } from "@lib/db/client";
5 import { HINTS, HUNT_SLUGS, type HuntSlug } from "@lib/hunt/content";
6
7 function isHuntSlug(slug: string): slug is HuntSlug {
8     return (HUNT_SLUGS as readonly string[]).includes(slug);
9 }

```



```

10
11 /**
12  * Buy the next hint.
13  *
14  * The increment is a CONDITIONAL update on the current hintsUsed, so two taps
15  * on a slow connection cost one hint rather than two. gradeHunt reads this
16  * field to reduce the award, and a double-charge is money the team never spent.
17  */
18 export async function POST(req: Request) {
19   const session = await getSession();
20   if (!session) return NextResponse.json({ error: "Not signed in" }, { status: 401 });
21
22   const { slug } = (await req.json().catch(() => ({}))) as { slug?: string };
23   if (!slug || !isHuntSlug(slug)) {
24     return NextResponse.json({ error: "Unknown puzzle" }, { status: 404 });
25   }
26
27   const teamId = new ObjectId(session.teamId);
28   const [challenges, progress] = await Promise.all([
29     collections.challenges(),
30     collections.huntProgress(),
31   ]);
32
33   const row = await progress.findOne({ teamId, challengeSlug: slug });
34   if (!row) return NextResponse.json({ error: "Not unlocked" }, { status: 403 });
35   if (row.solvedAt) return NextResponse.json({ error: "Already solved" }, { status: 409 });
36
37   const doc = await challenges.findOne({ type: "hunt", slug });
38   const costs: number[] = doc?.config.hintCosts ?? [];
39   const texts = HINTS[slug];
40
41   if (row.hintsUsed >= costs.length) {
42     return NextResponse.json({ error: "No hints left" }, { status: 409 });
43   }
44
45   const result = await progress.updateOne(
46     { teamId, challengeSlug: slug, hintsUsed: row.hintsUsed },
47     { $set: { hintsUsed: row.hintsUsed + 1 } }
48   );
49   if (result.modifiedCount === 0) {
50     return NextResponse.json({ error: "Try again" }, { status: 409 });
51   }
52
53   return NextResponse.json({
54     text: texts[row.hintsUsed],
55     hintsUsed: row.hintsUsed + 1,
56     cost: costs[row.hintsUsed],
57   });
58 }

```

THE CONDITIONAL UPDATE

The increment matches on the *current* `hintsUsed` and checks that a row was actually modified. Two taps on a slow connection therefore cost one hint, not two — and an over-charge is points the team never agreed to spend, which `gradeHunt` deducts silently.

Verified live with five concurrent purchases: exactly two succeeded, three got a conflict, and the count landed on two.

Note also what this route was fixing. `gradeHunt` had always deducted hint costs, but *nothing in the codebase ever incremented `hintsUsed`*. Hints were free, and the grader's deduction code had never once executed.

6 The shell and the puzzle contract

One route hosts all four puzzles. The shell owns the session, the answer box, hint purchasing and the chrome; a puzzle renders and calls `onSolve(code)`.

6.1 The contract

src/app/hunt/registry.tsx

33 lines

```
1 import type { ComponentType } from "react";
2 import dynamic from "next/dynamic";
3 import CaesarCipher from "../puzzles/CaesarCipher";
4 import SixtyFourGrid from "../puzzles/SixtyFourGrid";
5 import OctaviusCircuit from "../puzzles/OctaviusCircuit";
6
7 export interface PuzzleProps {
8   config: Record<string, unknown>;
9   onSolve: (code: string) => void;
10 }
11
12 export interface HuntPuzzle {
13   slug: string;
14   title: string;
15   Component: ComponentType<PuzzleProps>;
16 }
17
18 /**
19  * Only the room loads three.js, and only in the browser. Importing it
20  * statically would put a 3D engine in the bundle of a puzzle that is a text
21  * box and a slider.
22  */
23 const MysteryRoom = dynamic(() => import("../puzzles/MysteryRoom"), {
24   ssr: false,
25   loading: () => <p className="p-8 text-center text-paper-white/70">Entering the room...</p>,
26 });
27
28 export const REGISTRY: Record<string, HuntPuzzle> = {
29   "hunt-cipher": { slug: "hunt-cipher", title: "Caesar Cipher", Component: CaesarCipher },
30   "hunt-grid": { slug: "hunt-grid", title: "64 Grid", Component: SixtyFourGrid },
31   "hunt-circuit": { slug: "hunt-circuit", title: "Octavius Circuit", Component: OctaviusCircuit },
32   "hunt-room": { slug: "hunt-room", title: "Mystery Room", Component: MysteryRoom },
33 };
```

WHY THIS BOUNDARY MATTERS

There is exactly one place in the entire event that submits an answer. Four separate pages would mean four places to get the anti-cheat wrong, and three of them would be subtly different.

Only the Mystery Room loads three.js, through `next/dynamic`. A cipher that is a text box and a slider should not ship a 3D engine.

6.2 The shell

src/app/hunt/HuntShell.tsx

213 lines

```
1 "use client";
2
3 import { useCallback, useEffect, useState } from "react";
4 import { REGISTRY } from "../registry";
5
6 interface PuzzleState {
7   slug: string;
8   title: string;
9   points: number;
10  solved: boolean;
11  hintsUsed: number;
12  hintCount: number;
13  config: Record<string, unknown>;
14 }
15
16 /**
17  * The only thing in the hunt that submits an answer.
18  *
19  * Puzzles call onSolve(code) and know nothing about the network, their own
20  * answerHash, or how points work. That is deliberate: the anti-cheat decision
21  * is implemented once here rather than four times with three different bugs.
22  */
23 export default function HuntShell() {
24   const [puzzles, setPuzzles] = useState<PuzzleState[]>([]);
25   const [active, setActive] = useState<string | null>(null);
26   const [answer, setAnswer] = useState("");
27   const [hint, setHint] = useState<string | null>(null);
28   const [message, setMessage] = useState<string | null>(null);
29   const [busy, setBusy] = useState(false);
30
31   const load = useCallback(async () => {
32     const res = await fetch("/api/hunt/progress");
33     if (!res.ok) return;
34     const body = await res.json();
35     setPuzzles(body.puzzles ?? []);
36   }, []);
37
38   useEffect(() => {
39     void load();
40   }, [load]);
41
42   const current = puzzles.find((p) => p.slug === active) ?? null;
43
44   async function submit(code: string) {
45     if (!current || busy) return;
46     setBusy(true);
47     setMessage(null);
48     try {
49       const res = await fetch("/api/submit", {
50         method: "POST",
51         headers: { "content-type": "application/json" },
52         body: JSON.stringify({ event: "hunt", challengeSlug: current.slug, payload: code }),
53       });
54       const body = await res.json();
55
56       // A non-2xx (expired session, bad payload, server error) is not a
57       // wrong answer - body has no `correct` field at all in that shape, so
58       // treating it as falsy-correct would tell a team "Not that one." for a
59       // problem that has nothing to do with their answer. Surface the
60       // server's own message instead, same as buyHint() does below.
61       if (!res.ok) {
```

```

62     setMessage(body.error ?? "Something went wrong - try again");
63     return;
64 }
65
66 setMessage(body.correct ? `Solved - ${body.points} points` : "Not that one.");
67 setAnswer("");
68 await load();
69 } catch {
70     // fetch itself rejected - dropped wifi, not a server response at all.
71     // The team needs to know their tap did not reach the server so they
72     // retry rather than assume a correct answer was marked wrong.
73     setMessage("Couldn't reach the server - check your connection and try again");
74 } finally {
75     setBusy(false);
76 }
77 }
78
79 async function buyHint() {
80     if (!current) return;
81     const res = await fetch("/api/hunt/hint", {
82         method: "POST",
83         headers: { "content-type": "application/json" },
84         body: JSON.stringify({ slug: current.slug }),
85     });
86     const body = await res.json();
87     if (res.ok) {
88         setHint(body.text);
89         await load();
90     } else {
91         setMessage(body.error ?? "No hint available");
92     }
93 }
94
95 if (!active) {
96     return (
97         <main className="relative min-h-dvh overflow-hidden px-5 py-10">
98             <div
99                 aria-hidden="true"
100                 className="pointer-events-none absolute inset-0"
101                 style={{
102                     background:
103                         "radial-gradient(ellipse at 50% 0%, var(--web-blue-dark) 0%, transparent 60%)",
104                 }}
105             />
106             <div className="relative mx-auto max-w-3xl">
107                 <p className="font-body text-[0.7rem] uppercase tracking-[0.25em] text-glitch-cyan">
108                     XPLORE&apos;26
109                 </p>
110                 <h1 className="display-title chromatic mt-1 text-5xl text-paper-white">
111                     Treasure Hunt
112                 </h1>
113                 <p className="mt-3 text-sm text-paper-white/60">
114                     Four puzzles, any order. Solve one to reveal its code.
115                 </p>
116
117                 <ul className="mt-8 grid gap-3">
118                     {puzzles.map((p) => (
119                         <li key={p.slug}>
120                             <button
121                                 onClick={() => {
122                                     setActive(p.slug);
123                                     setHint(null);
124                                     setMessage(null);
125                                     setAnswer("");
126                                 }}
127                             className="halftone panel relative flex w-full items-center justify-between p-4

```

```

text-left transition-transform hover:-translate-y-0.5"
128         >
129         <span className="font-semibold text-paper-white">{p.title}</span>
130         <span className="text-sm text-paper-white/60">
131             {p.solved ? "solved" : `${p.points} pts`}
132         </span>
133     </button>
134 </li>
135     )}
136     {puzzles.length === 0 && (
137         <li className="text-sm text-paper-white/40">Loading puzzles...</li>
138     )}
139 </ul>
140 </div>
141 </main>
142 );
143 }
144
145 const entry = REGISTRY[current!.slug];
146 const Puzzle = entry.Component;
147
148 return (
149     <main className="relative min-h-dvh overflow-hidden px-5 py-10">
150         <div
151             aria-hidden="true"
152             className="pointer-events-none absolute inset-0"
153             style={{
154                 background:
155                     "radial-gradient(ellipse at 50% 0%, var(--web-blue-dark) 0%, transparent 60%)",
156             }}
157         />
158         <div className="relative mx-auto max-w-5xl">
159             <button
160                 onClick={() => setActive(null)}
161                 className="mb-4 text-sm text-paper-white/60 hover:text-paper-white"
162             >
163                 ← All puzzles
164             </button>
165             <h2 className="display-title text-3xl text-paper-white">{current!.title}</h2>
166
167             <div className="my-6">
168                 <Puzzle config={current!.config} onSolve={(code) => setAnswer(code)} />
169             </div>
170
171             <div className="flex flex-wrap items-center gap-3">
172                 <input
173                     value={answer}
174                     onChange={(e) => setAnswer(e.target.value)}
175                     placeholder="Enter the code"
176                     autoComplete="off"
177                     spellCheck={false}
178                     className="border-2 border-paper-white/20 bg-ink-black/60 px-3 py-2 font-mono uppercase
text-paper-white outline-none transition-colors focus:border-glitch-cyan"
179                 />
180                 <button
181                     onClick={() => void submit(answer)}
182                     disabled={busy || !answer.trim() || current!.solved}
183                     className="comic-btn"
184                 >
185                     Submit
186                 </button>
187                 <button
188                     onClick={() => void buyHint()}
189                     disabled={current!.hintsUsed >= current!.hintCount || current!.solved}
190                     className="comic-btn comic-btn-cyan"
191                 >

```

```

192     Hint ({current!.hintsUsed}/{current!.hintCount})
193   </button>
194 </div>
195
196   {hint && (
197     <p className="anim-pop mt-4 border-l-2 border-glitch-cyan bg-paper-white/5 p-3 text-paper-
white/90">
198       {hint}
199     </p>
200   )}
201   {message && (
202     <p
203       className={`anim-pop mt-3 font-semibold ${
204         current!.solved ? "text-glitch-cyan" : "text-signal-wrong"
205       }`}
206     >
207       {message}
208     </p>
209   )}
210 </div>
211 </main>
212 );
213 }

```

THREE OUTCOMES, NOT TWO

`submit()` distinguishes a correct answer, a wrong answer, and a request that failed. An earlier version ignored `res.ok`, so an expired session or a 500 rendered as "Not that one." — telling a team their correct answer was wrong, at an event, with a coordinator watching. A dropped connection showed nothing at all.

6.3 Making the contract enforceable

The rule "puzzles never fetch and never import content.ts" was documented and unenforced, which meant it held exactly as long as everyone remembered it. It is now a lint error.

eslint.config.mjs

49 lines

```

1 import { defineConfig, globalIgnores } from "eslint/config";
2 import nextVitals from "eslint-config-next/core-web-vitals";
3 import nextTs from "eslint-config-next/typescript";
4
5 const eslintConfig = defineConfig([
6   ...nextVitals,
7   ...nextTs,
8   // Override default ignores of eslint-config-next.
9   globalIgnores([
10     // Default ignores of eslint-config-next:
11     ".next/**",
12     "out/**",
13     "build/**",
14     "next-env.d.ts",
15   ]),
16   // Puzzle components render and call onSolve(code) — that's the whole
17   // contract. They must never fetch (only HuntShell submits) and must never
18   // import @/lib/hunt/content (it carries hints and is server/shell-only;
19   // @/lib/hunt/codes is the one client-safe hunt module). This was enforced
20   // by convention only; make the two invariants a lint error so Task 7/8
21   // can't drift from them by accident.
22   {
23     files: ["src/app/hunt/puzzles/**/*.ts", "src/app/hunt/puzzles/**/*.tsx"],
24     rules: {
25       "no-restricted-imports": [

```

```

26     "error",
27     {
28         paths: [
29             {
30                 name: "@lib/hunt/content",
31                 message:
32                     "Puzzles never import hunt content (it carries hints/answerHash). Use @lib/hunt/codes
if you need a client-safe code, or lift the logic into HuntShell.",
33             },
34         ],
35     },
36 ],
37 "no-restricted-globals": [
38     "error",
39     {
40         name: "fetch",
41         message:
42             "Puzzles never fetch. Render, then call onSolve(code) - HuntShell is the only thing that
submits.",
43     },
44 ],
45 },
46 },
47 ]);
48
49 export default eslintConfig;

```

CONVENTION VERSUS CONSTRAINT

This was added when a reviewer pointed out that nothing stopped the next two tasks from eroding the boundary. Both of those tasks then built puzzles against it without incident. A rule a machine checks is worth more than a rule in a comment.

7 The flat puzzles

Three components. All thin — the logic is in `src/lib/hunt/` and already tested, so these render state and forward clicks.

7.1 Caesar Cipher

`src/app/hunt/puzzles/CaesarCipher.tsx`

48 lines

```
1 "use client";
2
3 import { useEffect, useState } from "react";
4 import { shift } from "@lib/hunt/caesar";
5 import { CODES } from "@lib/hunt/codes";
6 import type { PuzzleProps } from "../registry";
7
8 /**
9  * The ciphertext plus a shift dial. Dragging re-renders the plaintext live, so
10  * the mechanic explains itself and nobody needs instructions read aloud over a
11  * PA system.
12  */
13 export default function CaesarCipher({ config, onSolve }: PuzzleProps) {
14   const ciphertext = String(config.ciphertext ?? "");
15   const [by, setBy] = useState(0);
16   const plain = shift(ciphertext, -by);
17
18   // The regex only DETECTS that the plaintext currently reads as "code
19   // <WORD>" at the dial's current position - it must never be the thing
20   // handed to onSolve. What goes up is always CODES.cipher, matching
21   // SixtyFourGrid/OctaviusCircuit/MysteryRoom, which all hand onSolve their
22   // CODES.* constant rather than anything derived from displayed content. If
23   // a coordinator reworded CIPHER.plaintext's trailing word (spec §2.6)
24   // without updating codes.ts, deriving the word from here would let the
25   // puzzle "solve" client-side while the server rejects the mismatched
26   // answer - see the content.test.ts assertion that keeps the two in sync.
27   useEffect(() => {
28     const m = plain.match(/code\s+([a-z]+)\s*$/i);
29     if (m) onSolve(CODES.cipher);
30   }, [plain, onSolve]);
31
32   return (
33     <div>
34       <p className="panel p-4 font-mono text-lg tracking-wider break-words text-paper-white">
35         {plain || "..."}
36       </p>
37       <label className="mt-4 block text-sm text-paper-white/70">Shift: {by}</label>
38       <input
39         type="range"
40         min={0}
41         max={25}
42         value={by}
43         onChange={(e) => setBy(Number(e.target.value))}
44         className="w-full accent-[var(--spider-red)]"
45       />
46     </div>
47   );
48 }
```


IT HANDS UP THE CONSTANT

The regex detects that the plaintext reads correctly; what reaches `onSolve` is `CODES.cipher`. Earlier it passed the regex capture, which meant rewording the content without editing `codes.ts` would make the puzzle visibly solve while the server rejected the answer — and the spec explicitly invites coordinators to reword content.

7.2 64 Grid

src/app/hunt/puzzles/SixtyFourGrid.tsx

180 lines

```
1 "use client";
2
3 import { useEffect, useState } from "react";
4 import { GRID_COLOURS, isAnagram, lettersFor, type GridCell } from "@lib/hunt/grid";
5 import { CODES } from "@lib/hunt/codes";
6 import type { PuzzleProps } from "../registry";
7
8 /**
9  * Contrast helpers for the grid cells.
10  *
11  * A single fixed ink colour cannot read against all eight swatches – purple
12  * (#7209b7) and blue (#4361ee) are dark enough that black text falls to
13  * ~2.1:1 and ~3.4:1, well under WCAG AA's 4.5:1 for body text, and these are
14  * letters a team has to read off a projector under a clock. Rather than
15  * touch the palette (the eight colours are load-bearing for the puzzle),
16  * each swatch picks whichever of the two design-system ink tones –
17  * ink-black or paper-white – actually reads better against it.
18  */
19 function srgbToLinear(channel: number): number {
20   const c = channel / 255;
21   return c <= 0.03928 ? c / 12.92 : Math.pow((c + 0.055) / 1.055, 2.4);
22 }
23
24 function relativeLuminance(hex: string): number {
25   const n = hex.replace("#", "");
26   const r = parseInt(n.slice(0, 2), 16);
27   const g = parseInt(n.slice(2, 4), 16);
28   const b = parseInt(n.slice(4, 6), 16);
29   return 0.2126 * srgbToLinear(r) + 0.7152 * srgbToLinear(g) + 0.0722 * srgbToLinear(b);
30 }
31
32 function contrastRatio(l1: number, l2: number): number {
33   const lighter = Math.max(l1, l2);
34   const darker = Math.min(l1, l2);
35   return (lighter + 0.05) / (darker + 0.05);
36 }
37
38 const INK_BLACK_LUM = relativeLuminance("#0A0A0A");
39 const PAPER_WHITE_LUM = relativeLuminance("#F2EFE9");
40
41 interface TextTone {
42   colour: string;
43   haloRgba: string;
44   ratio: number;
45 }
46
47 function textToneFor(hex: string): TextTone {
48   const lum = relativeLuminance(hex);
49   const onBlack = contrastRatio(lum, INK_BLACK_LUM);
50   const onWhite = contrastRatio(lum, PAPER_WHITE_LUM);
51   return onBlack >= onWhite
52     ? { colour: "var(--ink-black)", haloRgba: "rgba(242, 239, 233, 0.65)", ratio: onBlack }
53     : { colour: "var(--paper-white)", haloRgba: "rgba(10, 10, 10, 0.65)", ratio: onWhite };
```

```

54 }
55
56 // Computed once against the fixed eight-colour palette, not per render.
57 const CELL_TEXT_TONE = GRID_COLOURS.map(textToneFor);
58
59 /**
60  * Three equations give a number; that number mod 8 selects one of the eight
61  * swatches below. Picking a swatch isolates its eight scattered cells (the
62  * other 56 dim out) and shows their letters so the team can unscramble them.
63  *
64  * The grid is built server-side at seed time (see scripts/seed-hunt.ts) –
65  * config.gridCells arrives already shuffled. This component never sees the
66  * eight source words, only the letter/colour pairs, so there is nothing here
67  * for a curious team to read out of devtools.
68  */
69 export default function SixtyFourGrid({ config, onSolve }: PuzzleProps) {
70   const equations = Array.isArray(config.equations) ? (config.equations as string[]) : [];
71   const cells = Array.isArray(config.gridCells) ? (config.gridCells as GridCell[]) : [];
72
73   const [picked, setPicked] = useState<number | null>(null);
74   const [guess, setGuess] = useState("");
75
76   const letters = picked === null ? [] : lettersFor(cells, picked);
77
78   // The only thing that may reach onSolve is the real reveal code – never
79   // whatever the player happened to type. isAnagram(guess, letters) only
80   // proves the guess uses the picked colour's eight letters; it says nothing
81   // about their order, so it must never gate onSolve on its own. Typing the
82   // scrambled letters back unchanged (the first thing anyone tries) would
83   // otherwise satisfy it and hand the answer box a string the server is
84   // guaranteed to reject.
85   const normalizedGuess = guess.toUpperCase().replace(/[^A-Z]/g, "");
86   const solved = normalizedGuess.length > 0 && normalizedGuess === CODES.grid;
87
88   // Kept as feedback only: "you have the right cells, wrong order" is useful
89   // mid-solve signal that doesn't leak the code, since every one of the
90   // 8! orderings of those letters satisfies it, not just the real word.
91   const rightLettersWrongOrder =
92     picked !== null && !solved && letters.length > 0 && isAnagram(guess, letters.join(""));
93
94   // In an effect, not during render: calling the parent's setState from a
95   // render body is a React error, and it would fire on every keystroke.
96   useEffect(() => {
97     if (solved) onSolve(CODES.grid);
98   }, [solved, onSolve]);
99
100  return (
101    <div className="grid gap-6 md:grid-cols-2">
102      <div className="panel p-4">
103        <h3 className="font-semibold text-paper-white">Solve these</h3>
104        <ol className="mt-2 list-decimal pl-5 text-sm text-paper-white/90">
105          {equations.map((e) => (
106            <li key={e} className="my-1">
107              {e}
108            </li>
109          ))}
110        </ol>
111        <p className="mt-3 text-sm text-paper-white/70">
112          Add your three answers. The total modulo 8 is the colour's position below.
113        </p>
114
115        <div className="mt-4 flex flex-wrap gap-2">
116          {GRID_COLOURS.map((c, i) => (
117            <button
118              key={c}
119              type="button"

```

```

120         onClick={() => {
121             setPicked(i);
122             setGuess("");
123         }}
124         aria-label={`Colour ${i}`}
125         aria-pressed={picked === i}
126         style={{ background: c, color: CELL_TEXT_TONE[i].colour }}
127         className={`flex h-9 w-9 items-center justify-center rounded border-2 border-ink-black
text-xs font-bold transition-transform ${
128             picked === i ? "ring-2 ring-glitch-cyan" : "hover:-translate-y-0.5"
129         }`}
130     >
131         {i}
132     </button>
133 }}
134 </div>
135
136 {picked !== null && (
137     <div className="anim-pop mt-4">
138         <p className="font-mono text-xl tracking-[0.3em] text-glitch-cyan">
139             {letters.join(" ")}
140         </p>
141         <input
142             value={guess}
143             onChange={(e) => setGuess(e.target.value)}
144             placeholder="Unscramble them"
145             autoComplete="off"
146             spellCheck={false}
147             className="mt-2 w-full border-2 border-paper-white/20 bg-ink-black/60 px-3 py-2 font-
mono uppercase text-paper-white outline-none transition-colors focus:border-glitch-cyan"
148         />
149         {rightLettersWrongOrder && (
150             <p className="anim-pop mt-2 text-sm text-paper-white/60">
151                 Right letters - try a different order.
152             </p>
153         )}
154     </div>
155 )}
156 </div>
157
158 <div className="halftone panel relative grid grid-cols-8 gap-1 p-3">
159     {cells.map((cell, i) => {
160         const tone = CELL_TEXT_TONE[cell.colour];
161         return (
162             <div
163                 key={i}
164                 style={{
165                     background: GRID_COLOURS[cell.colour],
166                     color: tone.colour,
167                     textShadow: `0 0 3px ${tone.haloRgba}`,
168                 }}
169                 className={`flex aspect-square items-center justify-center rounded text-sm font-bold
transition-opacity ${
170                     picked !== null && cell.colour !== picked ? "opacity-25" : ""
171                 }`}
172             >
173                 {cell.letter}
174             </div>
175         );
176     })}
177 </div>
178 </div>
179 );
180 }

```

TWO THINGS MEASURED, NOT EYEBALLED

Contrast. The letters were a fixed dark ink on all eight swatches, which fails WCAG AA on six of them — purple at 2.1:1. The ink is now chosen per swatch by luminance. Compute the ratios; the palette looks fine and is not.

The solve condition. It once accepted any anagram of the shown letters, so retyping the scramble unchanged — the most natural first move — filled the answer box with a wrong string that the server then rejected.

7.3 Octavius Circuit

src/app/hunt/puzzles/OctaviusCircuit.tsx

81 lines

```
1 "use client";
2
3 import { useEffect, useState } from "react";
4 import { isConnected, openings, rotate, type Board, type Dir } from "@lib/hunt/circuit";
5 import { CODES } from "@lib/hunt/codes";
6 import type { PuzzleProps } from "../registry";
7
8 /** Draw a tile's open sides as four stubs from the centre. */
9 function TileGlyph({ dirs }: { dirs: Dir[] }) {
10   return (
11     <svg viewBox="0 0 40 40" className="h-full w-full">
12       <circle cx="20" cy="20" r="4" fill="currentColor" />
13       {dirs.includes(0) && <rect x="18" y="0" width="4" height="20" fill="currentColor" />}
14       {dirs.includes(1) && <rect x="20" y="18" width="20" height="4" fill="currentColor" />}
15       {dirs.includes(2) && <rect x="18" y="20" width="4" height="20" fill="currentColor" />}
16       {dirs.includes(3) && <rect x="0" y="18" width="20" height="4" fill="currentColor" />}
17     </svg>
18   );
19 }
20
21 /** Runtime guard so a malformed config can't crash the puzzle on a bad cast. */
22 function isBoard(value: unknown): value is Board {
23   return (
24     typeof value === "object" &&
25     value !== null &&
26     typeof (value as Board).w === "number" &&
27     typeof (value as Board).h === "number" &&
28     Array.isArray((value as Board).tiles)
29   );
30 }
31
32 /**
33  * config.board arrives already scrambled (see scripts/seed-hunt.ts, which
34  * asserts it isn't already solved before it's ever stored). Clicking a tile
35  * rotates it 90 degrees; the board lights up only once isConnected(board) is
36  * true - a mutual, source-to-sink path, not merely tiles that look joined.
37  */
38 export default function OctaviusCircuit({ config, onSolve }: PuzzleProps) {
39   const initial: Board = isBoard(config.board) ? config.board : { w: 0, h: 0, tiles: [] };
40   const [board, setBoard] = useState<Board>(initial);
41   const done = isConnected(board);
42
43   // In an effect, not during render: calling the parent's setState from a
44   // render body is a React error, and it would fire on every rotation.
45   useEffect(() => {
46     if (done) onSolve(CODES.circuit);
47   }, [done, onSolve]);
48
49   function turn(i: number) {
50     if (board.tiles[i].kind === "empty" || done) return;
51     setBoard((b) => ({ ...b, tiles: b.tiles.map((t, k) => (k === i ? rotate(t) : t)) }));
```

```

52   }
53
54   return (
55     <div className="panel p-4">
56       <div
57         className="grid gap-1"
58         style={{ gridTemplateColumns: `repeat(${board.w}, minmax(0, 1fr))`, maxWidth: 480 }}
59       >
60         {board.tiles.map((t, i) => (
61           <button
62             key={i}
63             type="button"
64             onClick={() => turn(i)}
65             disabled={t.kind === "empty"}
66             className={`aspect-square border-2 p-1 transition-colors
67               ${t.kind === "empty" ? "border-transparent" : "border-paper-white/15 hover:border-paper-
white/40"}
68               ${t.kind === "source" ? "text-spider-red" : ""}
69               ${t.kind === "sink" ? "text-web-blue-light" : ""}
70               ${done ? "anim-pop border-glitch-cyan text-glitch-cyan" : ""}}`}}
71         >
72           {t.kind !== "empty" && <TileGlyph dirs={openings(t)} />}
73         </button>
74       )]}
75     </div>
76     <p className="mt-3 text-sm text-paper-white/70">
77       {done ? "Circuit closed." : "Click a tile to rotate it. Connect the red source to the blue
sink."}
78     </p>
79   </div>
80 );
81 }

```

8 The Mystery Room

React Three Fiber. The camera stands still and turns; clicking a prop lifts it toward the camera and spins it so its face can be read. Five props carry the code between them.

No walking, no collision — deliberately. A player wedged in a wall on stage is unrecoverable, and nothing about the puzzle needs movement.

8.1 The manifest, and why the art can arrive later

src/lib/hunt/manifest.ts

38 lines

```
1 /**
2  * Where the five inspectable props sit, and what each one carries.
3  *
4  * `model: null` renders a primitive stand-in with the clue on a face - that
5  * is what makes the room playable and testable before any art exists.
6  * `Prop` (src/app/hunt/puzzles/MysteryRoomProp.tsx) branches on this field:
7  * a non-null value loads that path as a GLB via `Model`
8  * (MysteryRoomModel.tsx), which owns its own Suspense fallback and an error
9  * boundary that fails loudly rather than quietly falling back to the box.
10 * So swapping in real art really is a manifest edit - put the GLB in
11 * `public/`, point `model` at its path - with no change needed to either
12 * component.
13 *
14 * Array order here IS the reveal-code order. MysteryRoom.tsx splits
15 * CODES.room (@/lib/hunt/codes) into ROOM_MANIFEST.length equal fragments and
16 * shows fragment `i` on prop `i` - it never assembles the code from the order
17 * a player finds these in, only from this fixed order. ROOM.clues in
18 * @/lib/hunt/content is a copy of that same split, kept only so a unit test
19 * can assert the two never drift apart; it is NOT seeded into the challenge
20 * config and nothing at runtime reads it. Keep that lockstep invariant in
21 * mind before reordering this array or changing CODES.room's length.
22 */
23 export interface PropSlot {
24   id: string;
25   label: string;
26   model: string | null;
27   position: [number, number, number];
28   rotation: [number, number, number];
29   scale: number;
30 }
31
32 export const ROOM_MANIFEST: PropSlot[] = [
33   { id: "p1", label: "Desk drawer", model: null, position: [-2.2, 0.9, -3.0], rotation: [0, 0.4, 0],
34     scale: 1 },
35   { id: "p2", label: "Shelf box", model: null, position: [2.4, 1.4, -2.6], rotation: [0, -0.5, 0],
36     scale: 1 },
37   { id: "p3", label: "Floor crate", model: null, position: [-1.4, 0.4, 1.6], rotation: [0, 1.1, 0],
38     scale: 1 },
39   { id: "p4", label: "Wall panel", model: null, position: [3.1, 2.0, 0.8], rotation: [0, -1.2, 0],
40     scale: 1 },
41   { id: "p5", label: "Old terminal", model: null, position: [0.2, 1.0, -4.1], rotation: [0, 0.05, 0],
42     scale: 1 },
43 ];
```

PRIMITIVES SHIP

`model`: null renders a primitive stand-in, so the room is fully playable and testable before any GLB exists. Swapping in real art is a manifest edit.

That claim was false when first written: the component never read `slot.model`, so pointing a manifest entry at a GLB silently changed nothing — while the comment asserted otherwise. An artist would have gone debugging Blender for a bug in our loader.

8.2 The room

src/app/hunt/puzzles/MysteryRoom.tsx

188 lines

```
1 "use client";
2
3 import { useEffect, useMemo, useRef, useState, type MutableRefObject } from "react";
4 import { Canvas, useFrame, useThree } from "@react-three/fiber";
5 import { ROOM_MANIFEST } from "@lib/hunt/manifest";
6 import { CODES } from "@lib/hunt/codes";
7 import Prop from "../MysteryRoomProp";
8 import type { PuzzleProps } from "../registry";
9
10 /** Pixels of total pointer travel between pointerdown and pointerup before a gesture counts as a
look-drag rather than a click. */
11 const DRAG_THRESHOLD_PX = 6;
12
13 /**
14  * Camera stands still and turns — never walks.
15  *
16  * There is no movement and no collision here on purpose: a player wedged in
17  * a wall on stage, in front of everyone, is unrecoverable. Clamps below are
18  * carried from a vanilla three.js prototype that had already tuned them to
19  * keep the room framed: yaw +/-0.85 rad, pitch -0.42 to 0.32 rad, drag
20  * sensitivity 0.0026. Free-look would let a player end up staring at a blank
21  * ceiling with no idea how to get back.
22  *
23  * Also tracks total pointer travel for the current down-to-up gesture into
24  * `dragRef`, shared with every Prop. A browser's native "click" fires based
25  * on where pointerup lands, not on distance travelled since pointerdown, so
26  * without this a look-drag that starts and ends over the same prop would
27  * register as a pick. `moved` is set once travel crosses
28  * `DRAG_THRESHOLD_PX` and only reset on the next pointerdown — never on
29  * pointerup — so it is still correct by the time the click that follows
30  * pointerup is dispatched.
31  */
32 function LookRig({ dragRef }: { dragRef: MutableRefObject<{ moved: boolean }> }) {
33   const { camera, gl } = useThree();
34   const yaw = useRef(0);
35   const pitch = useRef(0);
36   const drag = useRef<{ x: number; y: number } | null>(null);
37   const gestureStart = useRef<{ x: number; y: number } | null>(null);
38
39   useEffect(() => {
40     const el = gl.domElement;
41     const down = (e: PointerEvent) => {
42       drag.current = { x: e.clientX, y: e.clientY };
43       gestureStart.current = { x: e.clientX, y: e.clientY };
44       dragRef.current.moved = false;
45     };
46     const up = () => {
47       drag.current = null;
48       gestureStart.current = null;
49     };

```

```

50     const move = (e: PointerEvent) => {
51         if (!drag.current) return;
52         yaw.current = Math.max(-0.85, Math.min(0.85, yaw.current - (e.clientX - drag.current.x) *
0.0026));
53         pitch.current = Math.max(-0.42, Math.min(0.32, pitch.current - (e.clientY - drag.current.y) *
0.0026));
54         drag.current = { x: e.clientX, y: e.clientY };
55
56         if (gestureStart.current) {
57             const dx = e.clientX - gestureStart.current.x;
58             const dy = e.clientY - gestureStart.current.y;
59             if (Math.hypot(dx, dy) > DRAG_THRESHOLD_PX) dragRef.current.moved = true;
60         }
61     };
62     el.addEventListener("pointerdown", down);
63     window.addEventListener("pointerup", up);
64     window.addEventListener("pointermove", move);
65     return () => {
66         el.removeEventListener("pointerdown", down);
67         window.removeEventListener("pointerup", up);
68         window.removeEventListener("pointermove", move);
69     };
70 }, [gl, dragRef]);
71
72 useFrame(() => {
73     camera.position.set(0, 2.55, 4.8);
74     camera.rotation.set(pitch.current, yaw.current, 0, "YXZ");
75 });
76 return null;
77 }
78
79 /** Four walls and a floor, lit by one overhead spot. No art yet – just enough geometry to read as a
room. */
80 function Shell() {
81     return (
82         <group>
83             <mesh rotation={[-Math.PI / 2, 0, 0]} position={[0, 0, -1.5]} receiveShadow>
84                 <planeGeometry args={[14, 12]} />
85                 <meshStandardMaterial color="#1a1d24" roughness={0.95} />
86             </mesh>
87             <mesh position={[0, 3, -6]} receiveShadow>
88                 <planeGeometry args={[14, 6]} />
89                 <meshStandardMaterial color="#20242d" roughness={0.98} />
90             </mesh>
91             <mesh rotation={[0, Math.PI / 2, 0]} position={[-6, 3, -1.5]} receiveShadow>
92                 <planeGeometry args={[12, 6]} />
93                 <meshStandardMaterial color="#1c2029" roughness={0.98} />
94             </mesh>
95             <mesh rotation={[0, -Math.PI / 2, 0]} position={[6, 3, -1.5]} receiveShadow>
96                 <planeGeometry args={[12, 6]} />
97                 <meshStandardMaterial color="#1c2029" roughness={0.98} />
98             </mesh>
99         </group>
100     );
101 }
102
103 /**
104  * The five on-face fragments, purely for display.
105  *
106  * These are NOT read from the server. `api/hunt/progress` used to ship a
107  * `clues` array straight out of the seeded config – `["AR","CH","IV","ES","88"]`
108  * – and every client received the whole reveal code, pre-assembled, on page
109  * load, before the room was ever opened: `.join("")` was the answer. The leak
110  * check in scripts/verify-hunt.ts missed it because it substring-matches the
111  * whole code against the served JSON, and the code was fragmented into
112  * two-character pieces that never appear as a contiguous match.

```



```

113 *
114 * CODES.room is client-safe by design (see codes.ts and spec §2.1) – the
115 * anti-cheat model is a team typing the code while a coordinator watches, not
116 * hiding it from the bundle. So this derives the on-face text FROM that
117 * already-public constant, in the browser, splitting it into
118 * ROOM_MANIFEST.length equal pieces in manifest order. Nothing about the
119 * fragments ever passes through a network response.
120 */
121 function fragmentsOf(code: string, count: number): string[] {
122   if (code.length % count !== 0) {
123     throw new Error(`Reveal code "${code}" does not split evenly into ${count} fragments`);
124   }
125   const len = code.length / count;
126   return Array.from({ length: count }, (_, i) => code.slice(i * len, (i + 1) * len));
127 }
128
129 export default function MysteryRoom({ onSolve }: PuzzleProps) {
130   const clues = useMemo(() => fragmentsOf(CODES.room, ROOM_MANIFEST.length), []);
131   const [held, setHeld] = useState<string | null>(null);
132   const [found, setFound] = useState<string[]>([]);
133   // Shared between LookRig and every Prop – see LookRig's doc comment.
134   const dragRef = useRef({ moved: false });
135
136   const complete = found.length === ROOM_MANIFEST.length;
137
138   // In an effect, not during render: calling the parent's setState from a
139   // render body is a React error, and it would fire on every pick. What goes
140   // up is always the CODES.room constant, never anything reconstructed from
141   // per-prop state – matching CaesarCipher/SixtyFourGrid/OctaviusCircuit,
142   // which all hand onSolve their CODES.* constant rather than derived text.
143   useEffect(() => {
144     if (complete) onSolve(CODES.room);
145   }, [complete, onSolve]);
146
147   function pick(id: string) {
148     setHeld((h) => (h === id ? null : id));
149     setFound((f) => (f.includes(id) ? f : [...f, id]));
150   }
151
152   return (
153     <div>
154       <div className="panel h-[520px] w-full overflow-hidden">
155         <Canvas shadows camera={{ fov: 48, position: [0, 2.55, 4.8] }}>
156           <color attach="background" args={['#05070b']} />
157           <fogExp2 attach="fog" args={['#05070b', 0.02]} />
158           <ambientLight intensity={0.25} />
159           <spotLight
160             position={[0, 4.6, 0]}
161             angle={0.9}
162             penumbra={0.6}
163             intensity={40}
164             color="#ffd9a0"
165             castShadow
166           />
167         <LookRig dragRef={dragRef} />
168         <Shell />
169         {ROOM_MANIFEST.map((slot, i) => (
170           <Prop
171             key={slot.id}
172             slot={slot}
173             clue={clues[i] ?? "?"}
174             held={held === slot.id}
175             found={found.includes(slot.id)}
176             onPick={() => pick(slot.id)}
177             dragRef={dragRef}
178           />

```

```

179     })}
180   </Canvas>
181 </div>
182 <p className="mt-3 text-sm text-paper-white/70">
183   Drag to look around. Click an object to pick it up and read its face. Found {found.length}
of{" " }
184   {ROOM_MANIFEST.length}.
185 </p>
186 </div>
187 );
188 }

```

MANIFEST ORDER, NOT DISCOVERY ORDER

The code assembles from the manifest, so a player who finds prop three first gets the same answer as one who finds prop one first. This is structural rather than tested-into-place: `found` only gates completeness and never feeds the string.

8.3 Props, and failing loudly

src/app/hunt/puzzles/MysteryRoomProp.tsx

99 lines

```

1  "use client";
2
3  import { useMemo, useRef, useState, type MutableRefObject } from "react";
4  import { useFrame } from "@react-three/fiber";
5  import { Text } from "@react-three/drei";
6  import { Vector3 } from "three";
7  import type { Group } from "three";
8  import type { PropSlot } from "@lib/hunt/manifest";
9  import Model from "../MysteryRoomModel";
10
11  interface Props {
12    slot: PropSlot;
13    clue: string;
14    held: boolean;
15    found: boolean;
16    onPick: () => void;
17    /** Shared with LookRig - see the comment on the click handler below. */
18    dragRef: MutableRefObject<{ moved: boolean }>;
19  }
20
21  /** Where a held prop floats: just in front of the fixed camera. */
22  const HELD_AT = new Vector3(0, 1.7, 2.6);
23
24  /**
25   * One inspectable object.
26   *
27   * Held props drift toward the camera and spin slowly so the clue face comes
28   * round without the player having to fight a drag control. Left as pure
29   * mouse-drag, reading a small label on a small object on a projector is
30   * genuinely hard.
31   *
32   * `slot.model === null` renders the primitive stand-in (the only case that
33   * exists today, since no GLB art has shipped yet). A non-null `slot.model`
34   * renders that GLB via `Model` (see MysteryRoomModel.tsx), which owns its
35   * own loading and error states - so swapping in real art is purely a
36   * manifest edit, and a broken path fails loudly instead of quietly showing
37   * a box.
38   */
39  export default function Prop({ slot, clue, held, found, onPick, dragRef }: Props) {
40    const ref = useRef<Group>(null);

```

```

41  const [hovered, setHovered] = useState(false);
42  const home = useMemo(() => new Vector3(...slot.position), [slot.position]);
43
44  useFrame(() => {
45    const g = ref.current;
46    if (!g) return;
47
48    if (held) {
49      g.position.lerp(HELD_AT, Math.min(1, delta * 4));
50      g.rotation.y += delta * 0.6;
51    } else {
52      g.position.lerp(home, Math.min(1, delta * 4));
53      g.rotation.y = slot.rotation[1];
54    }
55  });
56
57  return (
58    <group
59      ref={ref}
60      position={slot.position}
61      rotation={slot.rotation}
62      scale={slot.scale * (hovered && !held ? 1.08 : 1)}
63      onClick={e => {
64        e.stopPropagation();
65        // A native "click" fires on wherever pointerup lands, regardless of
66        // how far the pointer travelled since pointerdown - so a look-drag
67        // that starts and ends over this same prop would otherwise register
68        // as a pick. dragRef.moved is set by LookRig once total travel
69        // during the current down-to-up gesture crosses a small threshold;
70        // a click that follows a real drag is suppressed here rather than
71        // picked up.
72        if (dragRef.current.moved) return;
73        onPick();
74      }}
75      onPointerOver={e => {
76        e.stopPropagation();
77        setHovered(true);
78      }}
79      onPointerOut={() => setHovered(false)}
80    >
81      {slot.model === null ? (
82        <mesh castShadow>
83          <boxGeometry args={[0.6, 0.6, 0.6]} />
84          <meshStandardMaterial
85            color={found ? "#2a9d8f" : hovered ? "#e63946" : "#8d99ae"}
86            roughness={0.7}
87          />
88        </mesh>
89      ) : (
90        <Model path={slot.model} />
91      )}
92      {held && (
93        <Text position={[0, 0, 0.31]} fontSize={0.18} color="#0f1115" anchorX="center">
94          {clue}
95        </Text>
96      )}
97    </group>
98  );
99 }

```

```

1  "use client";
2
3  import { Component, Suspense, type ReactNode } from "react";

```

```

4 import { useGLTF, Text } from "@react-three/drei";
5
6 /**
7  * Loads a manifest-provided GLB in place of the primitive stand-in.
8  *
9  * Two failure modes this exists to make loud rather than silent:
10 *
11 * - Still loading: `useGLTF` suspends. The Suspense boundary is scoped to
12 *   this one model, not shared across the Canvas - a slow or large GLB
13 *   delays only its own prop, never the other four, which is the whole
14 *   point of keeping props independent in the first place.
15 * - Broken or missing path: caught by `ModelErrorBoundary` below, which
16 *   renders an unmistakable magenta wireframe marker carrying the offending
17 *   path as a floating label, and logs the same path plus the underlying
18 *   error to the console. Silently falling back to the primitive box here
19 *   was the exact bug being fixed: if the manifest says a GLB belongs on a
20 *   prop, something must say so loudly the moment it doesn't load, or
21 *   whoever wired up the model spends their afternoon in Blender instead of
22 *   here.
23 */
24 function GLTFMesh({ path }: { path: string }) {
25   const { scene } = useGLTF(path);
26   return <primitive object={scene} />;
27 }
28
29 /** Distinct on purpose from every other prop state (default grey, hovered red, found teal) so a
broken model can never be mistaken for a normal one. */
30 function BrokenModelMarker({ path }: { path: string }) {
31   return (
32     <group>
33       <mesh>
34         <boxGeometry args={[0.6, 0.6, 0.6]} />
35         <meshBasicMaterial color="#ff00ff" wireframe />
36       </mesh>
37       <Text position={[0, 0.5, 0]} fontSize={0.09} color="#ff00ff" anchorX="center" maxWidth={1.4}>
38         {`MODEL FAILED\n${path}`}
39       </Text>
40     </group>
41   );
42 }
43
44 /** Faint wireframe, deliberately unlike both the solved-state colours and the error marker, so "still
loading" never reads as "broken" or "done". */
45 function LoadingPlaceholder() {
46   return (
47     <mesh>
48       <boxGeometry args={[0.6, 0.6, 0.6]} />
49       <meshBasicMaterial color="#8d99ae" wireframe transparent opacity={0.35} />
50     </mesh>
51   );
52 }
53
54 interface BoundaryProps {
55   path: string;
56   children: ReactNode;
57 }
58 interface BoundaryState {
59   error: Error | null;
60 }
61
62 class ModelErrorBoundary extends Component<BoundaryProps, BoundaryState> {
63   state: BoundaryState = { error: null };
64
65   static getDerivedStateFromError(error: Error): BoundaryState {
66     return { error };
67   }

```

```

68
69 componentDidCatch(error: Error): void {
70   // Deliberately loud: a manifest entry pointing at a bad path must never
71   // fail in silence, or the next person to touch this assumes their GLB
72   // export is broken instead of the path in manifest.ts.
73   console.error(`[MysteryRoom] failed to load model "${this.props.path}":`, error);
74 }
75
76 render() {
77   if (this.state.error) return <BrokenModelMarker path={this.props.path} />;
78   return this.props.children;
79 }
80 }
81
82 export default function Model({ path }: { path: string }) {
83   return (
84     <ModelErrorBoundary path={path}>
85       <Suspense fallback={<LoadingPlaceholder />}>
86         <GLTFMesh path={path} />
87       </Suspense>
88     </ModelErrorBoundary>
89   );
90 }

```

A BAD MODEL PATH MUST BE VISIBLE

A missing GLB renders a magenta wireframe labelled with the failing path, plus a console error — never a silent fallback to the primitive. The error boundary sits *outside* Suspense, and each prop has its own, so one broken model cannot blank the room.

9 Verification

One script drives the whole event against a running server: creates teams, solves puzzles, buys hints, checks standings, and releases everything it claimed.

9.1 Running it

```
npm run dev -- -p 3100

# reseed first - the script claims coins and solves puzzles
npx tsx --env-file=.env.local scripts/seed-hunt.ts
npx tsx --env-file=.env.local scripts/verify-hunt.ts
```

THE PRINCIPLE IT IS BUILT ON

Assert the things that fail silently. A crash announces itself; you will find it without a test. What you will not find is a leaked answer, an accepted wrong code, a hint that does not deduct, or a puzzle submittable without being unlocked. Every one of those leaves the app working perfectly and the event unfair.

scripts/verify-hunt.ts

530 lines

```
1 /**
2  * Drive the whole treasure hunt against a running server.
3  *
4  *   npm run dev -- -p 3100
5  *   npx tsx --env-file=.env.local scripts/seed-hunt.ts
6  *   npx tsx --env-file=.env.local scripts/verify-hunt.ts
7  *
8  * RESEED BEFORE EVERY RUN. This script is not idempotent by nature - it claims
9  * coins, buys hints, and solves puzzles, all of which are permanent writes.
10 * A naive version of this script would hard-code a few coins and re-use the
11 * same teams on every run, and a SECOND run against teams the FIRST run
12 * already solved puzzles for would then fail checks that are actually fine -
13 * "no puzzle starts solved" looks identical whether it's really broken or
14 * whether it's just the same team from last time. There is no way to tell
15 * those apart from the output alone.
16 *
17 * This script avoids the stale-team trap by never hard-coding a coin: every
18 * team below claims whatever the database says is still unclaimed (see
19 * `freshCoin()`), so every run gets brand-new teams with brand-new,
20 * never-touched progress rows - a second run without reseeding will mostly
21 * still pass, because there's no stale team left around to collide with.
22 * Reseed before every run anyway: this script's expected numbers (85, 60, 100
23 * points; hint costs of 15 and 25) are only true of whatever `seed-hunt.ts`
24 * most recently wrote. If that file is edited - a hint cost changed, or (see
25 * the deliberate leak-proof this script was built to catch) a reveal code
26 * temporarily added to a challenge's config - the live challenge documents
27 * only reflect that edit once seed-hunt.ts has actually been re-run. Skip the
28 * reseed and this script checks the OLD config, silently.
29 *
30 * COINS ARE RELEASED, NOT SPENT. Claiming a coin in /api/enter is otherwise a
31 * one-way transition - nothing in the app ever unclaims one - and a coin is a
32 * physical printed disc: one left claimed by a test run is one a real team's
33 * disc silently fails to enter with at the door. So this script tracks every
34 * team it creates under five fixed, reserved names (EPHEMERAL_TEAM_NAMES) and
35 * releases them - coin, team, participant, progress, score events - in a
36 * `finally` around the whole run, whether it finishes or throws. And because
37 * those names are reserved for this script alone, `releaseEphemeralTeams`
```

```

38 * ALSO runs once at the very start, so a run that got killed before its own
39 * `finally` could fire (a hard process kill bypasses `finally` entirely) is
40 * healed by the next run rather than leaking a coin forever.
41 *
42 * The database also holds two OTHER "hunt"-type challenges - the pre-existing
43 * `clue-1` and `clue-2` from scripts/seed.ts - alongside the four `hunt-*`
44 * puzzles this script is actually about. Six challenges share `type: "hunt"`;
45 * only the four in HUNT_SLUGS are this script's concern, and every query
46 * below that touches the challenges collection is scoped accordingly.
47 *
48 * THE PRINCIPLE: ASSERT THE THINGS THAT FAIL SILENTLY. A crash announces
49 * itself - you will find it without a test. What you will not find is a
50 * leaked answer, an accepted wrong code, a hint that does not deduct, or a
51 * puzzle that can be submitted without being unlocked. Every one of those
52 * leaves the app working perfectly and the event unfair.
53 */
54 import { ObjectId } from "mongodb";
55 import { collections } from "../src/lib/db/client";
56 import { hashAnswer } from "../src/lib/auth/session";
57 import { HUNT_SLUGS } from "../src/lib/hunt/content";
58 import { CODES } from "../src/lib/hunt/codes";
59 import { formatCoin } from "../src/lib/quiz/avatars";
60 import { materialize } from "../src/lib/leaderboard/materialize";
61
62 const BASE = process.env.HUNT_BASE ?? "http://localhost:3100";
63
64 let passed = 0;
65 const failures: string[] = [];
66
67 function check(label: string, condition: boolean, detail?: unknown) {
68   if (condition) {
69     passed++;
70     console.log(` ✓ ${label}`);
71   } else {
72     failures.push(label);
73     console.log(` X ${label}`);
74     if (detail !== undefined) console.log(`      got: ${JSON.stringify(detail)}`);
75   }
76 }
77
78 /** Minimal cookie-carrying client - one per team, like one browser per team. */
79 class Client {
80   private cookie = "";
81   constructor(readonly label: string) {}
82
83   private async send(method: string, path: string, body?: unknown) {
84     const res = await fetch(`${BASE}${path}`, {
85       method,
86       headers: {
87         "content-type": "application/json",
88         ...(this.cookie ? { cookie: this.cookie } : {}),
89       },
90       body: body === undefined ? undefined : JSON.stringify(body),
91     });
92
93     const setCookie = res.headers.get("set-cookie");
94     if (setCookie) this.cookie = setCookie.split(";")[0];
95
96     const text = await res.text();
97     // eslint-disable-next-line @typescript-eslint/no-explicit-any
98     let json: any = null;
99     try {
100       json = JSON.parse(text);
101     } catch {
102       /* non-JSON error page */
103     }

```

```

104     return { status: res.status, body: json, raw: text };
105 }
106
107 get(path: string) {
108     return this.send("GET", path);
109 }
110 post(path: string, body?: unknown) {
111     return this.send("POST", path, body);
112 }
113 }
114
115 /**
116  * Coins are a finite, permanently-depleting pool – claim whichever is still
117  * free rather than hard-coding a number that an earlier run may already own.
118  */
119 async function freshCoin(): Promise<string> {
120     const coins = await collections.coins();
121     const doc = await coins.findOne({ teamId: null });
122     if (!doc) {
123         throw new Error(
124             "No unclaimed coins left in the pool of 60 – every coin has been claimed by " +
125             "an earlier run. seed-hunt.ts does not reset coins (they are shared with the " +
126             "quiz), so the fix is to reset the `coins` collection's teamId/claimedAt " +
127             "fields, or point HUNT_BASE at a fresh event database."
128         );
129     }
130     return formatCoin(doc._id);
131 }
132
133 interface Entered {
134     client: Client;
135     teamId: ObjectId;
136 }
137
138 /** Claim a fresh coin and register a brand-new team under it. */
139 async function enter(name: string): Promise<Entered> {
140     const coin = await freshCoin();
141     const client = new Client(name);
142     const res = await client.post("/api/enter", { coin, teamName: name });
143     if (res.status !== 200 || !res.body?.teamId) {
144         throw new Error(`Could not enter team "${name}" with coin ${coin}: ${JSON.stringify(res.body)}`);
145     }
146     return { client, teamId: new ObjectId(String(res.body.teamId)) };
147 }
148
149 /**
150  * The five ephemeral teams this run creates – one name each, always exactly
151  * these, never anything else. Earlier runs (before this fix) used the plain
152  * names "verify-hunt-a".."verify-hunt-e", and fifty teams already exist under
153  * those from that history; a sweep keyed on THAT name would catch them too,
154  * which is precisely what "match on the teams you created, not a name
155  * pattern that could catch something real" rules out – those fifty are
156  * pre-existing data in a shared database, and cleaning them up is not this
157  * script's call to make. This distinct, never-before-used name set is what
158  * makes a name-based sweep safe: going forward, nothing else is ever named
159  * one of these five things, so finding one IS finding this script's own
160  * leftover, with no ambiguity.
161  */
162 const EPHEMERAL_TEAM_NAMES = [
163     "verify-hunt:ephemeral:a",
164     "verify-hunt:ephemeral:b",
165     "verify-hunt:ephemeral:c",
166     "verify-hunt:ephemeral:d",
167     "verify-hunt:ephemeral:e",
168 ] as const;
169

```



```

170 /**
171  * Release every team this script has ever created, and the coin each
172  * claimed. Idempotent, and keyed only on EPHEMERAL_TEAM_NAMES above – so the
173  * same function serves two purposes:
174  *
175  * - called once at the very start of a run, it heals whatever an earlier
176  *   run left behind because it never reached its own cleanup (killed,
177  *   crashed, or threw when the coin pool ran out mid-flight)
178  * - called again in a top-level 'finally', it releases what THIS run just
179  *   created, whether the run finished cleanly or not
180  *
181  * A no-op when nothing matches, which is the common case for the second call.
182  */
183 async function releaseEphemeralTeams(): Promise<void> {
184   const teams = await collections.teams();
185   const coins = await collections.coins();
186   const participants = await collections.participants();
187   const progress = await collections.huntProgress();
188   const scoreEvents = await collections.scoreEvents();
189
190   const owned = await teams.find({ name: { $in: [...EPHEMERAL_TEAM_NAMES] } }).toArray();
191   if (owned.length === 0) return;
192
193   const ownedIds = owned.map((t) => t._id!);
194   await coins.updateMany({ teamId: { $in: ownedIds } }, { $set: { teamId: null, claimedAt: null } });
195   await participants.deleteMany({ teamId: { $in: ownedIds } });
196   await progress.deleteMany({ teamId: { $in: ownedIds } });
197   await scoreEvents.deleteMany({ teamId: { $in: ownedIds } });
198   await teams.deleteMany({ _id: { $in: ownedIds } });
199 }
200
201 async function main() {
202   console.log(`\nVerifying the treasure hunt against ${BASE}\n`);
203
204   const challenges = await collections.challenges();
205   const progress = await collections.huntProgress();
206
207   // Heal whatever a previous run left behind (killed, crashed, or exhausted
208   // the coin pool mid-flight) BEFORE spending any of this run's own coins –
209   // see releaseEphemeralTeams()'s docs above.
210   await releaseEphemeralTeams();
211
212   try {
213     await runChecks(challenges, progress);
214   } finally {
215     // Release this run's own teams and coins, whether the run above finished
216     // cleanly or threw partway through.
217     await releaseEphemeralTeams();
218   }
219
220   // — Summary —————
221   console.log("\n" + "-".repeat(62));
222   if (failures.length === 0) {
223     console.log(` ALL ${passed} CHECKS PASSED`);
224   } else {
225     console.log(` ${passed} passed, ${failures.length} FAILED:`);
226     failures.forEach((f) => console.log(`   X ${f}`));
227   }
228   console.log("-".repeat(62) + "\n");
229   process.exit(failures.length === 0 ? 0 : 1);
230 }
231
232 /** Everything that spends a coin or writes a probe row. Split out of main()
233  * purely so the coin-release finally above wraps it cleanly. */
234 async function runChecks(
235   challenges: Awaited<ReturnType<typeof collections.challenges>>,

```

```

236 progress: Awaited<ReturnType<typeof collections.huntProgress>>
237 ) {
238   // — Unlocking —————
239   console.log("— Unlocking —————");
240   const a = await enter(Ephemeral_Team_Names[0]);
241
242   const prog = await a.client.get("/api/hunt/progress");
243   check(
244     "a new team has all four puzzles unlocked",
245     prog.body?.puzzles?.length === HUNT_SLUGS.length,
246     prog.body
247   );
248   check(
249     "no puzzle starts solved",
250     Array.isArray(prog.body?.puzzles) && prog.body.puzzles.every((p: { solved: boolean }) => p.solved
=== false),
251     prog.body?.puzzles
252   );
253
254   // — The leak check —————
255   // This is the one that would otherwise be decoration if it never fails -
256   // see the "deliberately break it" step in the task report for proof that
257   // it does.
258   console.log("\n— The leak —————");
259   const servedRaw = JSON.stringify(prog.body ?? {});
260   const servedUpper = servedRaw.toUpperCase();
261   // Strip everything except letters and digits before searching. A plain
262   // substring search against the raw JSON is exactly what let the Mystery
263   // Room's reveal code through once already: seed-hunt.ts stored
264   // `clues: ["AR", "CH", "IV", "ES", "88"]`, which serialises to
265   // `clues: ["AR", "CH", "IV", "ES", "88"]` - the code split across five
266   // separate two-character JSON string elements, never appearing as one
267   // contiguous "ARCHIVES88" substring, so the old check reported green while
268   // the client received every fragment needed to assemble the answer with a
269   // single `.join("")`. Once quotes, brackets, commas and colons are removed,
270   // those five fragments collapse back into one contiguous run and a plain
271   // `.includes()` catches it - this is the fourth leak of this shape in this
272   // project, so the check itself has to close the class, not just this one
273   // instance of it.
274   const servedStripped = servedUpper.replace(/^[A-Z0-9]/g, "");
275   const codes = [CODES.cipher, CODES.grid, CODES.circuit, CODES.room];
276   check(
277     "no reveal code is served to the client by /api/hunt/progress, even fragmented across array
elements",
278     codes.every((code) => !servedStripped.includes(code)),
279     codes.filter((code) => servedStripped.includes(code))
280   );
281   check("no answerHash is served to the client", !servedRaw.includes("answerHash"));
282   check(
283     "no puzzle slug names its own reveal code",
284     HUNT_SLUGS.every((slug, i) => !slug.toUpperCase().includes(codes[i].slice(0, 4)))
285   );
286
287   // — Wrong then right —————
288   console.log("\n— Wrong then right —————");
289   const wrong = await a.client.post("/api/submit", {
290     event: "hunt",
291     challengeSlug: "hunt-cipher",
292     payload: "NOTITMATE",
293   });
294   check("a wrong code scores zero", wrong.body?.correct === false && wrong.body?.points === 0,
wrong.body);
295
296   const stillOpen = await a.client.get("/api/hunt/progress");
297   check(
298     "a wrong code does not mark the puzzle solved",

```

```

299     stillOpen.body?.puzzles?.find((p: { slug: string }) => p.slug === "hunt-cipher")?.solved ===
false,
300     stillOpen.body?.puzzles
301 );
302
303 const right = await a.client.post("/api/submit", {
304     event: "hunt",
305     challengeSlug: "hunt-cipher",
306     payload: CODES.cipher,
307 });
308 check(
309     "a correct code awards full points",
310     right.body?.correct === true && right.body?.points === 100,
311     right.body
312 );
313
314 const again = await a.client.post("/api/submit", {
315     event: "hunt",
316     challengeSlug: "hunt-cipher",
317     payload: CODES.cipher,
318 });
319 check(
320     "a solved puzzle rejects a second submission",
321     again.body?.correct === false && again.body?.meta?.reason === "already-solved",
322     again.body
323 );
324
325 // — Hints deduct —————
326 console.log("\n— Hints deduct —————");
327 const b = await enter(Ephemeral_Team_Names[1]);
328 const hint1 = await b.client.post("/api/hunt/hint", { slug: "hunt-grid" });
329 check("buying a hint returns its text", typeof hint1.body?.text === "string", hint1.body);
330 check("the first hint costs 15", hint1.body?.cost === 15, hint1.body);
331
332 const gridSolve = await b.client.post("/api/submit", {
333     event: "hunt",
334     challengeSlug: "hunt-grid",
335     payload: CODES.grid,
336 });
337 check(
338     "one hint reduces the award by exactly its cost (100 - 15 = 85)",
339     gridSolve.body?.correct === true && gridSolve.body?.points === 85,
340     gridSolve.body
341 );
342
343 const c = await enter(Ephemeral_Team_Names[2]);
344 await c.client.post("/api/hunt/hint", { slug: "hunt-circuit" });
345 await c.client.post("/api/hunt/hint", { slug: "hunt-circuit" });
346 const thirdHint = await c.client.post("/api/hunt/hint", { slug: "hunt-circuit" });
347 check("a third hint is refused", thirdHint.status === 409, thirdHint.status);
348
349 const circuitSolve = await c.client.post("/api/submit", {
350     event: "hunt",
351     challengeSlug: "hunt-circuit",
352     payload: CODES.circuit,
353 });
354 check(
355     "both hints reduce the award by their sum (100 - 15 - 25 = 60)",
356     circuitSolve.body?.correct === true && circuitSolve.body?.points === 60,
357     circuitSolve.body
358 );
359
360 // — The clamp, tested where it can actually engage —————
361 // With the real values - 100 points, hints of 15 and 25 - the clamp can
362 // NEVER fire (worst case 100 - 15 - 25 = 60), so asserting it against a real
363 // puzzle would pass while testing nothing. A synthetic challenge whose hint

```

```

364 // costs exceed its points is the only honest way to reach the clamp.
365 //
366 // The hint-purchase endpoint (/api/hunt/hint) only accepts the four real
367 // HUNT_SLUGS by design (src/app/api/hunt/hint/route.ts's isHuntSlug guard) -
368 // a synthetic slug would 404 there. That guard is what's under test for
369 // checks 9-12 above, not this one: check 13 is about the clamp arithmetic in
370 // gradeHunt, which reads hintsUsed off the progress row regardless of which
371 // door put it there. So hintsUsed is written directly here instead of
372 // purchased through the API.
373 console.log("\n— The clamp —————");
374 const CLAMP_SLUG = "hunt-clamp-probe";
375 // The submission pipeline tags every score event "<event>:<slug>" (see
376 // src/lib/submission/pipeline.ts's appendScore call) - for this probe,
377 // "hunt:hunt-clamp-probe". The probe's answer is correct by construction
378 // (that's the whole point - it needs to score, just zero, to prove the
379 // clamp), so this row really does get written every run, and it points at
380 // a challenge slug that won't exist a moment later. It's harmless (zero
381 // points, so no leaderboard effect) but it's the same accumulation
382 // pattern as everything else here, so it's swept the same way.
383 const CLAMP_SCORE_REASON = `hunt:${CLAMP_SLUG}`;
384
385 // Self-healing: delete by the fixed slug/reason BEFORE inserting, so a run
386 // that died after the insert and before its own cleanup is mopped up by
387 // the NEXT run rather than lingering. Wrapped in try/finally too, so an
388 // in-process failure (a thrown assertion, a rejected request) still
389 // cleans up immediately rather than waiting for that next run.
390 const cleanupClampProbe = async () => {
391   await progress.deleteMany({ challengeSlug: CLAMP_SLUG });
392   await challenges.deleteOne({ slug: CLAMP_SLUG });
393   await (await collections.scoreEvents()).deleteMany({ reason: CLAMP_SCORE_REASON });
394 };
395 await cleanupClampProbe();
396
397 try {
398   await challenges.insertOne({
399     type: "hunt",
400     slug: CLAMP_SLUG,
401     title: "Clamp probe (synthetic - verify-hunt.ts only)",
402     points: 10,
403     opensAt: null,
404     closesAt: null,
405     config: { answerHash: hashAnswer("PROBE"), hintCosts: [30] },
406   });
407
408   const d = await enter(EPHEMERAL_TEAM_NAMES[3]);
409   await progress.insertOne({
410     teamId: d.teamId,
411     challengeSlug: CLAMP_SLUG,
412     unlockedAt: new Date(),
413     solvedAt: null,
414     hintsUsed: 1, // one hint "used" at cost 30 - more than the challenge is worth
415   });
416
417   const clamped = await d.client.post("/api/submit", {
418     event: "hunt",
419     challengeSlug: CLAMP_SLUG,
420     payload: "PROBE",
421   });
422   check(
423     "an award clamps at zero rather than going negative (10 - 30 -> 0, not -20)",
424     clamped.body?.correct === true && clamped.body?.points === 0,
425     clamped.body
426   );
427 } finally {
428   await cleanupClampProbe();
429 }

```

```

430
431 // — Locked puzzles —————
432 console.log("\n— Locked puzzles —————");
433 const e = await enter(EPHEMERAL_TEAM_NAMES[4]);
434 await progress.deleteOne({ teamId: e.teamId, challengeSlug: "hunt-room" });
435 const locked = await e.client.post("/api/submit", {
436   event: "hunt",
437   challengeSlug: "hunt-room",
438   payload: CODES.room,
439 });
440 check(
441   "a puzzle with no progress row cannot be submitted",
442   locked.body?.correct === false && locked.body?.meta?.reason === "clue-not-unlocked",
443   locked.body
444 );
445
446 // — Standings —————
447 // The leaderboard is a materialized snapshot (leaderboard_snapshots),
448 // refreshed on demand only when the document is missing – once ANY run has
449 // ever asked for it, later GETs serve that same document forever, stale or
450 // not. Calling the materializer directly (as this script already reaches
451 // past the API for the clamp probe above) is the only way to get a snapshot
452 // that reflects what this run just wrote rather than whatever an earlier
453 // run happened to cache.
454 //
455 // Two synthetic teams score IDENTICAL points at different times. This is
456 // the only way to prove the SECOND sort key (time, ascending) actually
457 // runs – asserting on real teams' standings would pass even if the
458 // tiebreak were reversed or missing entirely, as long as no two teams ever
459 // happened to tie, which makes it a check that can't fail.
460 //
461 // This writes fake teams and fake score rows straight into the real
462 // database, so it gets the same two guards as the clamp probe above, for
463 // the same reason – except here it matters more: these two calls span an
464 // HTTP round trip to a server under active testing, and this is the real
465 // Azure Cosmos DB the event runs on. If this dies between the insert and
466 // the cleanup, "verify-hunt-standings-early"/"-late" holding 500 points
467 // each could show up on the projected scoreboard on the day.
468 //
469 // 1. DETERMINISTIC ids, not random ObjectIds. A random id, once orphaned,
470 //    can never be found again by anything. A fixed id can be deleted by
471 //    the next run even if this one never reaches its own cleanup.
472 // 2. Delete BEFORE inserting (self-healing across runs) AND clean up in
473 //    a 'finally' (so a thrown assertion or a failed request cleans up
474 //    immediately, without waiting for a next run at all).
475 console.log("\n— Standings —————");
476 const teams = await collections.teams();
477 const scoreEvents = await collections.scoreEvents();
478
479 const STANDINGS_PROBE_REASON = "verify-hunt:tie-probe";
480 const EARLY_TEAM_ID = new ObjectId("11111111111111111111111111111111");
481 const LATE_TEAM_ID = new ObjectId("22222222222222222222222222222222");
482
483 const cleanupStandingsProbe = async () => {
484   await scoreEvents.deleteMany({ reason: STANDINGS_PROBE_REASON });
485   await teams.deleteMany({ _id: { $in: [EARLY_TEAM_ID, LATE_TEAM_ID] } });
486 };
487 await cleanupStandingsProbe();
488
489 try {
490   await teams.insertMany([
491     { _id: EARLY_TEAM_ID, name: "verify-hunt-standings-early", createdAt: new Date() },
492     { _id: LATE_TEAM_ID, name: "verify-hunt-standings-late", createdAt: new Date() },
493   ]);
494   await scoreEvents.insertMany([
495     { teamId: EARLY_TEAM_ID, event: "hunt", points: 500, reason: STANDINGS_PROBE_REASON, at: new

```

```

Date("2026-01-01T00:00:00Z") },
496     { teamId: LATE_TEAM_ID, event: "hunt", points: 500, reason: STANDINGS_PROBE_REASON, at: new
Date("2026-01-01T00:05:00Z") },
497   ]};
498
499   const snapshot = await materialize("hunt");
500   const rankOf = (id: ObjectId) => snapshot.rows.findIndex((r) => r.teamId === id.toString());
501   const earlyRank = rankOf(EARLY_TEAM_ID);
502   const lateRank = rankOf(LATE_TEAM_ID);
503   check(
504     "standings rank on points then time",
505     earlyRank !== -1 && lateRank !== -1 && earlyRank < lateRank,
506     { earlyRank, lateRank }
507   );
508   check(
509     "standings never rank a lower-points team above a higher-points one",
510     snapshot.rows.every((r, i, arr) => i === 0 || arr[i - 1].points >= r.points),
511     snapshot.rows.map((r) => r.points)
512   );
513
514   const board = await a.client.get(`/api/leaderboard?event=hunt`);
515   check(
516     "the leaderboard API serves that same ranking",
517     Array.isArray(board.body?.rows) &&
518       board.body.rows.some((r: { teamId: string }) => r.teamId === EARLY_TEAM_ID.toString()),
519     board.body?.rows?.length
520   );
521 } finally {
522   await cleanupStandingsProbe();
523   await materialize("hunt"); // restore the live snapshot without the probe rows, even after a
failure above
524 }
525 }
526
527 main().catch((e) => {
528   console.error(e);
529   process.exit(1);
530 });

```

9.2 Checks that can actually fail

Two of these deserve attention, because the obvious version of each proves nothing.

The clamp. With 100 points and hints costing 15 and 25, the award can never go negative — so asserting "it clamps at zero" against a real puzzle passes while testing nothing. It runs against a synthetic challenge whose hints cost more than it is worth.

The leak. It strips non-alphanumerics from the response before searching. The earlier version substring-matched the whole code and reported green while the Mystery Room shipped its answer as `["AR", "CH", "IV", "ES", "88"]` — five fragments that join into the code. A guard written for exactly one bug missed it on a technicality.

PROVE A NEW CHECK CAN FAIL

Deliberately break the thing it watches, watch it go red, then fix it. Three separate checks in this project passed for the wrong reason and were caught only by someone doing this.

9.3 Leave the database as you found it

The script claims coins from a finite pool of sixty printed discs shared with the real event. Early versions never released them: after roughly ten runs, 56 of 60 coins were held by test teams, and any of those discs handed to a real team at the door would have been silently refused.

It now tracks everything it creates and releases it in a top-level `try/finally`, with deterministic identifiers and a sweep before it starts — so a run killed mid-flight is cleaned up by the next.

TEST DATA ON A REAL LEADERBOARD

Those teams also carried score events. A materialised hunt leaderboard showed 32 rows of test teams, top-ranked, with the real teams below all of them. That is the failure everyone in the room sees at once.

10 What went wrong

The most useful part of this document. Every one of these passed a code review before it was caught, and most were caught by measuring rather than reading.

10.1 Nobody could play

Severity: total. Fifteen real teams, zero progress rows, a spinner that never resolved and no error shown.

Rows were created in `/api/enter` and deleted by `seed-hunt.ts`. Nothing backfilled them. One task owned the seed, another owned entry, and no task owned the lifecycle. Nine per-task reviews passed because each piece was correct in isolation.

Found by: a whole-branch review that queried the live database instead of reading the diff.

Fixed by: upserting the rows on every progress read, so any page load heals the gap.

10.2 Four leaks of the same shape

The answer reaching the browser, four times, in four disguises:

WHERE	HOW IT LEAKED
Quiz Connections	Puzzle slug was <code>connect-python</code> and tiles were served from <code>/quiz/connect-python-1.svg</code> — the answer was in the network tab
Grid seed	<code>gridWords</code> stored the eight source words, one of which is the answer
Mystery Room	<code>clues: ["AR", "CH", "IV", "ES", "88"]</code> served on page load; join them
The check itself	Substring-matched the whole code, so fragments slipped past it

THE PATTERN

Every time, the thing that leaked was *derived* data that looked innocent in isolation. A slug is not an answer. A word list is not an answer. Five two-character strings are not an answer. They only become one when someone joins them, and nobody reviewing a diff joins them.

The durable fix is to search the actual response for the actual code, with punctuation stripped, and to prove that check fails.

10.3 The scramble that took four rounds

`scramble()` must never return an already-solved board. Getting there took four attempts, and each failure was in the *evidence* rather than the code.

ROUND	CLAIM	REALITY
Original	50-seed test passes	The fixture made the guarantee hold by accident; other boards broke 116 times in 2000
Fix 1	"exhausting the guard is now impossible"	False in general — a hub topology broke it 131 times in 2,000,000
Fix 2	Hub regression test proves it	The board could not connect under any of its 4096 rotations; the test passed against the bug
Fix 3	Swept 20,000 seeds, caught nothing	Reported as "variance" and shipped
Fix 4	—	Deterministic hub assertion + 200,000 seeds + proof of failure at seed 10,564

THE LESSON

A green test suite is evidence only if the tests can fail. Three consecutive rounds shipped a test that passed against the bug it was written to catch, and each round the report said it worked.

The fix that finally held asserts the *property* first — that the board is genuinely a hub — before sweeping seeds. If the fixture is wrong, that assertion fails immediately and loudly rather than passing for the wrong reason.

10.4 Hints that cost 40 points to mislead you

Both Mystery Room hints were factually false.

"Two are behind you when you start" — all five props are in front, and the camera's yaw is clamped to ± 0.85 rad, so *behind you is unreachable by construction*. A team following that hint could not perform the action it described.

"The fragments read in the order the objects are numbered" — describes a manual assembly step that does not exist; the component assembles the code and fills the box.

The grid's equations were worse: they asked for "Peter's floor number", which is defined nowhere in the app, the config, or the hints. A team buying both hints still could not solve it, and the unpunished fallback was to click all eight swatches and guess.

CONTENT IS NOT DECORATION

These cost 15 and 25 points each. Wrong content is a scoring bug that no test catches, because every test asserts the mechanism rather than the meaning. Someone has to read the hints as a player would.

10.5 Smaller ones, and what they share

- **Hints were free.** `gradeHunt` deducted hint costs from day one; nothing ever incremented `hintsUsed`. The deduction had never executed.
- **Failures read as wrong answers.** `submit()` ignored `res.ok`, so a 401 or 500 rendered as "Not that one."
- **The manifest did nothing.** The prop component never read `slot.model`, so pointing it at a GLB silently kept the placeholder.

- **The verifier consumed the event's coins.** Five per run, never released, from a pool of sixty physical discs.
- **CI never ran the tests.** Forty unit tests including a 200,000-seed sweep, executed only when a human remembered.

WHAT THEY HAVE IN COMMON

Not one of these throws. Every single one leaves the application working perfectly and the event subtly unfair or unrecoverable. That is the class of bug this kind of event generates, and it is why the verification script is the highest-value file in the project.

11 Running it

What the coordinator does, in order.

11.1 Before the doors open

1. **Scale Cosmos RU up.** The default throttles under load — deleting 55 documents in one call was enough to trigger error 16500. Thirty teams submitting at once will find it faster. Scale back down afterwards; it is billed hourly.
2. **Reseed, then run the verifier.** Both, in that order. A green run against the real deployment is the only evidence the thing in front of you works.
3. **Check the coin pool.** Every disc handed out must be unclaimed. A coin consumed by a test run is silently refused at the door, and the team has no idea why.
4. **Read the hints aloud.** As a player, not as the person who wrote them. That is how the false ones were caught.

11.2 When something breaks mid-event

Points are an append-only ledger, so almost any scoring mistake is recoverable without editing a total by hand.

PROBLEM	RECOVERY
A team's puzzles will not load	They reload the page — the upsert heals it
Content is wrong	Edit <code>content.ts</code> , reseed. Progress survives, because the upsert restores rows the reseed deleted
A team needs a retry	Delete their submission for that puzzle; the grader reopens it
A coin will not work	Set its <code>teamId</code> back to null

RESIST EDITING SCORES DIRECTLY
It is the one operation with no audit trail. Everything else leaves a record you can show a team that disputes a result.

12 Appendix — the remaining source

Every file the hunt needs that has not already been printed in full. With the listings above this is the complete implementation — 26 files.

Read the comments. A good deal of the reasoning in this guide is also recorded next to the code it explains, which is where it stays useful after the guide is filed away.

12.1 src/lib/hunt/grid.test.ts

src/lib/hunt/grid.test.ts

66 lines

```
1 import { describe, expect, it } from "vitest";
2 import { buildGrid, GRID_COLOURS, isAnagram, lettersFor } from "../grid";
3
4 const WORDS = [
5   "spiderly", "webbings", "villains", "symbiote",
6   "multiver", "gwenpool", "octopusx", "daybugle",
7 ];
8
9 describe("buildGrid", () => {
10   it("produces exactly 64 cells", () => {
11     expect(buildGrid(WORDS, 1).length).toBe(64);
12   });
13
14   it("gives every colour exactly 8 cells", () => {
15     const grid = buildGrid(WORDS, 1);
16     for (let c = 0; c < GRID_COLOURS.length; c += 1) {
17       expect(grid.filter((cell) => cell.colour === c).length).toBe(8);
18     }
19   });
20
21   it("scatters each word rather than leaving it in a run", () => {
22     const grid = buildGrid(WORDS, 1);
23     const positions = grid
24       .map((cell, i) => ({ ...cell, i }))
25       .filter((cell) => cell.colour === 0)
26       .map((cell) => cell.i);
27     const contiguous = positions.every((p, k) => k === 0 || p === positions[k - 1] + 1);
28     expect(contiguous).toBe(false);
29   });
30
31   it("is deterministic for a given seed", () => {
32     expect(buildGrid(WORDS, 42)).toEqual(buildGrid(WORDS, 42));
33   });
34
35   it("differs between seeds", () => {
36     expect(buildGrid(WORDS, 1)).not.toEqual(buildGrid(WORDS, 2));
37   });
38
39   it("rejects a word list that is not 8 words of 8 letters", () => {
40     expect(() => buildGrid(["short"], 1)).toThrow();
41     expect(() => buildGrid([...WORDS, "extraone"], 1)).toThrow();
42   });
43 });
44
45 describe("lettersFor", () => {
46   it("returns that colour's 8 letters and they anagram to the word", () => {
47     const grid = buildGrid(WORDS, 7);
48     const letters = lettersFor(grid, 3);
```

```

49     expect(letters.length).toBe(8);
50     expect(isAnagram(letters.join(""), WORDS[3])).toBe(true);
51   });
52 });
53
54 describe("isAnagram", () => {
55   it("ignores order and case", () => {
56     expect(isAnagram("SPIDER", "redips")).toBe(true);
57   });
58
59   it("rejects different letter multisets", () => {
60     expect(isAnagram("spider", "spidee")).toBe(false);
61   });
62
63   it("rejects different lengths", () => {
64     expect(isAnagram("spider", "spiders")).toBe(false);
65   });
66 });

```

12.2 src/lib/hunt/content.test.ts

src/lib/hunt/content.test.ts

84 lines

```

1  import { describe, expect, it } from "vitest";
2  import { CIPHER, CIRCUIT, GRID, ROOM } from "../content";
3  import { CODES } from "../codes";
4  import { isConnected, rotate, type Board } from "../circuit";
5  import { ROOM_MANIFEST } from "../manifest";
6
7  /**
8   * These are the assertions that the puzzles' four different reveal-code
9   * derivations actually agree with CODES.* – see the doc comment at the top
10  * of content.ts and spec §2.6. Nothing about a puzzle component "looking
11  * solved" is proof it hands up the right code; these tests are that proof,
12  * so a coordinator reordering content.ts (per spec §2.6) without touching
13  * codes.ts fails a test instead of failing silently in front of a team.
14  */
15
16  describe("CIPHER content agrees with CODES.cipher", () => {
17    it("the plaintext's trailing word, uppercased, equals CODES.cipher", () => {
18      const m = CIPHER.plaintext.match(/code\s+([a-z]+)\s*$/i);
19      expect(m).not.toBeNull();
20      expect(m![1].toUpperCase()).toBe(CODES.cipher);
21    });
22  });
23
24  describe("GRID content agrees with CODES.grid", () => {
25    it("the three answers sum to targetColour mod 8", () => {
26      const sum = GRID.answers.reduce((a, b) => a + b, 0);
27      expect(sum % 8).toBe(GRID.targetColour);
28    });
29
30    it("the target colour's word, uppercased, equals CODES.grid", () => {
31      expect(GRID.words[GRID.targetColour].toUpperCase()).toBe(CODES.grid);
32    });
33  });
34
35  describe("ROOM content agrees with CODES.room", () => {
36    it("ROOM.clues concatenate to CODES.room", () => {
37      expect(ROOM.clues.join("")).toBe(CODES.room);
38    });
39
40    // MysteryRoom.tsx never reads ROOM.clues at runtime (nothing does – see
41    // manifest.ts's doc comment). It derives the on-screen fragments itself by

```

```

42 // splitting CODES.room into ROOM_MANIFEST.length equal pieces, in manifest
43 // order. ROOM.clues is only a checked-in copy of that same split so it
44 // can't silently drift from what the component actually shows - assert the
45 // real relationship, not the content file in isolation.
46 it("splitting CODES.room into ROOM_MANIFEST.length equal fragments reproduces ROOM.clues", () => {
47   expect(CODES.room.length % ROOM_MANIFEST.length).toBe(0);
48   const len = CODES.room.length / ROOM_MANIFEST.length;
49   const fragments = Array.from({ length: ROOM_MANIFEST.length }, (_, i) =>
50     CODES.room.slice(i * len, (i + 1) * len)
51   );
52   expect(fragments).toEqual(ROOM.clues);
53 });
54 };
55
56 describe("CIRCUIT.board is solvable", () => {
57   it("some rotation of every non-empty tile reaches isConnected", () => {
58     // The seed script only asserts the *scrambled* board is NOT already
59     // solved (see scripts/seed-hunt.ts) - the inverse of what matters here.
60     // A board whose topology can never be connected by any rotation would
61     // pass that check and seed silently; teams would then rotate tiles
62     // forever with no way to win. Brute-force every rotation of every
63     // non-empty tile on the pre-scramble board (7 tiles here, 4^7 = 16384
64     // combinations - trivial to exhaust) and require at least one to close
65     // the circuit.
66     const board = CIRCUIT.board;
67     const movable = board.tiles
68       .map((t, i) => i)
69       .filter((i) => board.tiles[i].kind !== "empty");
70
71     function solvable(from: Board, idx: number): boolean {
72       if (idx === movable.length) return isConnected(from);
73       const i = movable[idx];
74       let cur = from;
75       for (let r = 0; r < 4; r += 1) {
76         if (solvable(cur, idx + 1)) return true;
77         cur = { ...cur, tiles: cur.tiles.map((t, k) => (k === i ? rotate(t) : t)) };
78       }
79       return false;
80     }
81
82     expect(solvable(board, 0)).toBe(true);
83   });
84 });

```

12.3 src/app/hunt/page.tsx

src/app/hunt/page.tsx

14 lines

```

1 import { redirect } from "next/navigation";
2 import { getSession } from "@lib/auth/guard";
3 import HuntShell from "./HuntShell";
4
5 /**
6  * HUNT - reached via proxy rewrite from hunt.<domain>.
7  * The real event UI: four puzzles, a shared answer box, and the only place
8  * in this event that ever calls /api/submit.
9  */
10 export default async function HuntPage() {
11   const session = await getSession();
12   if (!session) redirect("/enter");
13   return <HuntShell />;
14 }

```